

LEPL - Kutaisi International University
School of Computer Science
Bachelor of Science Program in Computer Science

Giorgi Gogsadze

Revaz Gogvadze

Build a Large Language Model from scratch

The thesis is completed for obtaining the academic degree of
Bachelor of Science (BSc) in Computer Science

Walter F. Tichy

Professor

Kutaisi, 2026

Declaration

We, the authors of this bachelor's thesis, declare that the thesis is our own original work and does not contain any material previously published, submitted for publication, or presented for defense by other authors, unless properly referenced or cited in accordance with academic standards.

Giorgi Gogsadze, Revaz Goguadze

Kutaisi, 06.05.2026

Statement on the Use of AI

We, the authors of this bachelor's thesis, declare that the usage of AI, namely ChatGPT and Gemini, was limited strictly to grammar and syntax corrections and stylistic enhancements. We affirm that the core ideas, research, work and conclusions presented in this thesis are entirely our own original work.

Giorgi Gogsadze, Revaz Goguadze

Kutaisi, 06.05.2026

Contributions

During the semester Giorgi Gogsadze and Revaz Goguadze collaborated through in person and online meetings to research, plan, develop, code and test all the foundational components of a Large Language Model. After researching the principles of pretraining and implementing the primary phase, due to hardware limitations they agreed to use GPT-2 pretrained model and shift the focus to task-specific adaptation. This work was assigned into specialized areas: Giorgi Gogsadze developed the sentiment classification fine-tuning pipeline, and Revaz Goguadze designed the instruction fine-tuning framework for academic writing.

	Researching for LLM designs	Coding Tokenization and Attention mechanism	Structural implementation of GPT architecture	Pretraining the Model	Fine tuning for classification	Fine tuning for Instruction following
Giorgi Gogsadze	X	X	X	X	X	
Revaz Goguadze	X	X	X	X		X

Contribution Matrix

Abstract

This thesis is a practical study of how an LLM can be built from Scratch. It answers the question whether a useful LLM can be built and used locally with limited resources. The thesis starts as an expanded summary of Sebastian Raschka's book, Build a Large Language Model (from Scratch). While following the book's strategies, diagrams and foundational codes, all text is originally written by us and is extended by additional experiments, including all exercises given in the book and two independent projects. Despite being more compact, this document fully covers all essential materials required to understand and construct an LLM.

Our project describes all the necessary steps: converting raw text into vectors, exploring spatial and semantic relationships with the self-attention mechanism and building a GPT-2 architectural style to form the final model structure.

After constructing the architecture, the project shifts to a completely new stage of implementing our own practical work related to using the pretrained model. It will be fine-tuned several times, first for Sentiment Classification, which is a useful tool that illustrates the ability of the machine to distinguish emotions of humans through their text. After that, the model will be trained to fix grammar mistakes and rephrase texts in an academic style that illustrates the overall power of LLMs.

By applying all foundational components and connecting theoretical concepts with practical use cases, the thesis successfully demonstrates how a regular computer can be taught to use human language.

General keywords

Design, Coding, Large Language Models, Transformer Architecture, Pretraining, Parameter-Efficient Fine-Tuning, Measurement, Performance.

Table of Contents

1. Introduction	1
1.1 Core Concepts and Technical Insights.....	1
1.2 Real-World Relevance.....	2
1.3 Reflection	2
2. Generate Input Embeddings	3
2.1 Text to Tokens	4
2.2 Tokens to IDs	5
2.3 Data sampling with a sliding window	8
2.4 IDs to Token Embeddings	10
2.5 Token Embeddings to Input Embeddings	11
3. Code Attention Mechanism	13
3.1 Simplified self-attention without trainable weights	14
3.2 Self-attention with trainable weights	15
3.3 Causal attention that hides future words	17
3.4 Multi-head attention	18
4. Implement a GPT Model	20
4.1 Layer Normalization.....	22
4.2 Activation Functions	23
4.3 Feed-forward network.....	23
4.4 How Shortcut Connections Help.....	24
4.5 Transformer Block.....	25
4.6 Develop GPT Model and assess different configurations	26
4.7 Testing the GPT Model and Generating Text	27

5. Pretrain on Unlabeled Data.....	30
5.1 Evaluating Generative Text Models	30
5.2 Training and Validation Losses	32
5.3 LLM Training Loop Implementation	33
5.4 Improving the Training Loop	36
5.5 Decoding Strategies (temperature, top-k)	38
5.6 Saving and Loading Model Weights	40
5.7 Loading Pretrained GPT-2 Weights	42
6. Fine-tune for classification	44
6.1 Dataset Preparation	44
6.2 Data Loader Construction	46
6.3 Model Adaptation for Classification	48
6.4 Implement Evaluation Metrics	50
6.5 Model Fine-Tuning and Analysis	52
6.6 Discussion and Conclusion	55
7. Fine-tune to follow instructions	56
7.1 Model and Training Setup	57
7.2 Dataset Gathering.....	58
7.3 Sequential Training Design	59
7.4 Evaluation Protocol.....	60
7.5 Quantitative Analysis	61
7.6 Manual Testing and Qualitative Results	65
7.7 Specialization VS Generalization.....	67
8. Conclusion	68

8.1 Observations: Efficiency VS Effectiveness	69
8.2 Limitations of the Current Prototype.....	70
8.3 Final Summary	70
Bibliography.....	71
Appendices	72
Appendix A: Technical Specifications.....	72
Appendix B: Transformer Architecture	73
Appendix C: Byte Pair Encoding (BPE)	74
Appendix D: torch.nn.Embedding.....	75
Appendix E: The Architecture of Query, Key, and Value Representations	76
Appendix F: GELU VS RELU	77
Appendix G: Calculating the Loss	78
Appendix H: AdamW Optimizer	79
Appendix I: Parameter-Efficient Fine-Tuning with Low-Rank Adaptation.....	80
Appendix J: Token-F1 & Perplexity	81
Appendix K: BLEU & GLEU.....	83

1. Introduction

Artificial Intelligence (AI) and especially Large Language Models (LLMs) have become an integral part of modern life. Big companies and successful businesses increasingly seek greater automation and integration of LLMs into their systems. However, as demand continues to grow, people have to pay more and more for commercial large language models. Considering all these, experts believe that the local LLMs, which are either trained from scratch or fine-tuned from publicly available pre-trained models, will play an important role in the future. This thesis aims to summarize all the steps necessary for developing such models, starting with training and concluding with fine-tuning, while examining the advantages and limitations that may arise throughout the process to finally answer the question whether functional specialized LLM can be built on a local machine without massive data centers and resources.

This thesis is a summary of the book by Sebastian Raschka “Build a Large Language Model (from Scratch)”. It covers the same material in a more compact format, but also extends it with further explorations and original LLM implementation projects. Nevertheless, we hope it is understandable and enables the reader to build his/her own LLM.

1.1 Core Concepts and Technical Insights

Large Language Models (LLMs) are deep neural networks designed to understand, generate, and interact with human language. It includes the term “large” due to a massive training datasets and the billions of internal parameters it needs for working.

To better understand what the LLM is, first we need to understand how it is connected to or differentiated from broader concepts, starting from **AI**, which is an umbrella term for all the systems that can perform tasks requiring human-like intelligence. **Machine Learning** provides the algorithms, and **Deep Learning** uses neural networks to find data patterns. Finally, an **LLM** is a specialized application of Deep Learning dedicated to one thing - mastering language.

Developing a useful LLM requires a two-stage process:

- **Pretraining:** The model is trained on massive amounts of unlabeled text with the goal of predicting the next word. This phase creates a base Model.
- **Fine-Tuning:** The base model is further trained on smaller, labeled datasets adapted to specific, highly specialized skills.

These models rely on the Transformer architecture. See appendix B for details.

Remarkably, models trained simply to predict the next word develop complex, untaught skills. This “emergent behavior” happens due to the large amount of data and the algorithm’s ability to find patterns and connections within them. This allows for:

- **Zero-Shot Learning:** Performing a completely new task without prior examples.
- **Few-Shot Learning:** Performing a task after being provided with just a few examples.

1.2 Real-World Relevance

LLMs have changed today’s world and productivity through text generation, document summarization and translation. They accelerate software development by writing and debugging code and serve as an assistant even in specialized sectors like finance and medicine. Eventually, LLMs demonstrate ability to optimize efficiency and expertise in almost every field.

1.3 Reflection

Based on this initial research, we gained a solid foundational understanding of Large Language Models and the two-stage process it needs: massive pretraining and fine-tuning. Most importantly, we understand that an LLM's most impressive abilities are not directly programmed, rather emergent behaviors that arise naturally from the model learning deep statistical patterns in language simply by being trained to predict the next word.

So far, our understanding is theoretical. From now on our primary goal is to build a Large Language Model from scratch.

2. Generate Input Embeddings

The next goal in this process is to turn words into vectors and generate input embeddings we can feed to LLM, but in order to do so, first we need to understand what is the word embedding itself.

Word embeddings are numerical vector representations of words that can be processed by neural networks. Since text cannot be directly used for calculations, embeddings map each word to a continuous vector space, where words with similar meanings or contexts are located near one another. For example, “duck” and “goose” would have similar embeddings as they appear in the same contexts, shown in figure 2.1.

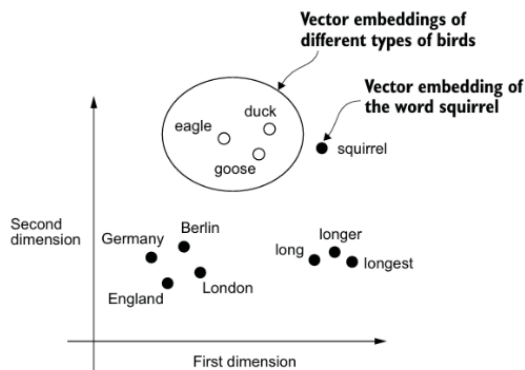


Fig. 2.1: two-dimensional word embeddings
(Source: [1], p. 20)

Before, the methods such as **Word2Vec** would learn embeddings by predicting surrounding words of a target word, capturing semantic relationships. In modern large language models (LLMs), embeddings are initialized randomly and are optimized during training. This makes them task-specific and more powerful than fixed pretrained embeddings. Of course, the size of embeddings varies, from hundreds to thousands of dimensions, trading off between efficiency (computational cost) and effectiveness (capturing richer meaning).

In order to fulfill the goal of this section, we need several steps to transform text into embedding vectors that serve as input to a GPT-like decoder-only transformer. We use [2] as a training data and start the process with tokenization.

2.1 Text to Tokens

Tokenization is the mechanism for splitting raw text into smaller pieces, which are called tokens. Tokens can be words, subwords, or even individual characters. It is a common preprocessing step when working with language models, since it makes text appear as something that models can operate on.

First, we split the training document into the list of words and punctuation characters.

for example, the sentence:

“The height of his glory”--that was what the women called it.

Should return the following list of tokens:

['"', 'The', 'height', 'of', 'his', 'glory', '"', '--', 'that', 'was', 'what', 'the', 'women', 'called', 'it', '.']

We can achieve this with the help of Python's `re` library:

```
import re
# Split text by whitespace, punctuation, and double dash
preprocessed = re.split(r'([,.;?_!"()\'| |--|\s])', raw_text)
# Remove empty strings and strip whitespace
preprocessed = [item.strip() for item in preprocessed if item.strip()]
```

The preprocessed list now contains the text's every punctuation mark and word. The models then require special tokens to deal with some context, for example:

- [BOS] beginning of sequence
- [EOS] end of sequence
- [PAD] padding

For our task, [EOS] (which is the same as `<|endoftext|>`) is the most important token. It marks the end of sequences. We can optionally use an `<|unk|>` token since the input could contain words not seen in the training data, but GPT models have Byte Pair Encoding (BPE) tokenizers that break words into subword units, so as we are going to use it, there is no need for `<|unk|>`. Check Appendix C for details about BPE.

2.2 Tokens to IDs

Now, as we have the tokens, it is time to map them to integer IDs. The implementation here is straightforward, take the preprocessed array and remove duplicates. This can be done by converting it into a set. After that sort the set and assign each token an integer corresponding to its position:

```
all_words = sorted(list(set(preprocessed)))
all_words.append("<|endoftext|>") # make sure special tokens are last
vocab = {token:integer for integer,token in enumerate(all_words)}
```

With this, we have a vocabulary containing token-ID pairs:

```
('!', 0)
...
('Her', 49)
('Hermia', 50)
...
('<|endoftext|>', 1130)
```

Next, we implement a text tokenizer class with encode and decode methods, which map tokens to IDs and vice versa, like shown in the picture:

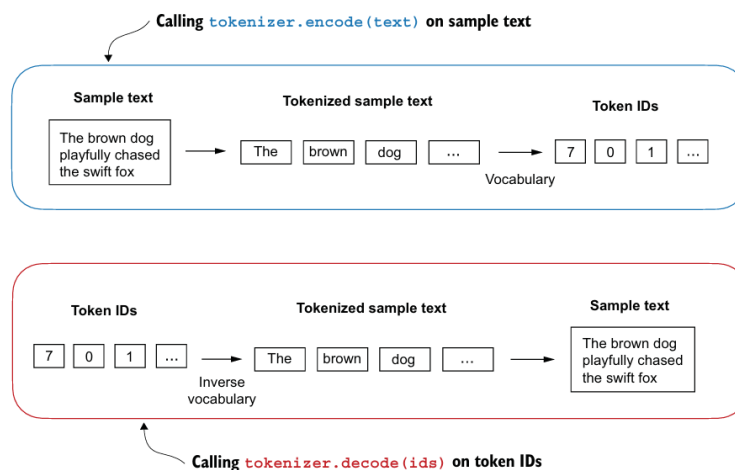


Fig. 2.2: Tokenizer (Source: [1], p. 28)

The encode method uses regular expression (`re.split`) to separate words from punctuation, strips whitespaces and converts these text tokens into their corresponding integer IDs from the vocabulary. In contrast, the decode method converts IDs back into words and joins them with spaces.

With simple test we can see that it works well for our text:

```
text = """"It's the last he painted, you know,"  
      Mrs. Gisburn said with pardonable pride.""""  
ids = tokenizer.encode(text)  
print(ids)
```

results in:

```
[1, 56, 2, 850, 988, 602, 533, 746, 5, 1126, 596, 5, 1, 67, 7, 38, 851, 1108, 754,  
793, 7]
```

And decoding:

```
print(tokenizer.decode(ids))
```

returns expected result:

```
" It' s the last he painted, you know," Mrs. Gisburn said with pardonable pride.
```

However, at this point there is no specific implementation for unknown words so we start using BPE (See the definition in Appendix C). The implementation of BPE is out of scope for this project, but we will use the tiktoken library.

```
from importlib.metadata import version  
import tiktoken  
# Load GPT-2 BPE tokenizer  
tokenizer = tiktoken.get_encoding("gpt2")  
# Sample text  
text = (  
    "Hello, how are you? <|endoftext|> Do you like"  
    " someunknownPlace."  
)  
integers = tokenizer.encode(text, allowed_special={"<|endoftext|>"})  
print(integers)
```

This prints all the IDs:

```
[15496, 11, 703, 389, 345, 30, 220, 50256, 2141, 345, 588, 617, 34680, 27271, 13]
```

To observe it better, let us see what each of these IDs corresponds to:

Hello

,

how

are

you

?

<|endoftext|>

Do

you

like

some

unknown

Place

.

It is clear that some words are divided into subwords, and some of them even have a space character merged with them. With decoding we can see that it ideally assembles the input.

```
strings = tokenizer.decode(integers)
```

```
print(strings)
```

```
Hello, do you like tea? <|endoftext|> In the sunlit terraces of someunknownPlace.
```

2.3 Data sampling with a sliding window

The next step in creating the embeddings for the LLM is to generate the input–target pairs required to teach an LLM how to predict the next word in a sequence. A common and effective method for creating these pairs is the “sliding window” approach.

This approach suggests that we walk through the whole text, take some number of tokens as an input (this amount is called max length or context size) and create a target by shifting the input by one position. Then, jump to other tokens and take them as an input. The number of tokens we jump over is called stride. It is good practice to make stride equal to max length as in this case we utilize the data set fully (we do not skip a single word) and avoid any overlap between the batches, which could otherwise increase overfitting. Figure 2.3 shows our goal.

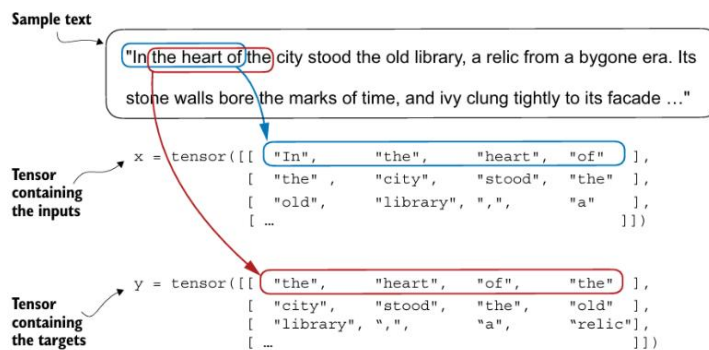


Fig. 2.3: data loader visualization
(Source: [1], p. 37)

There are words instead of token IDs for demonstration purposes

implementation of the already explained logic of sliding window approach is following:

```
class GPTDatasetV1(Dataset):
    def __init__(self, txt, tokenizer, max_length, stride):
        self.input_ids = []
        self.target_ids = []
        token_ids = tokenizer.encode(txt)
        for i in range(0, len(token_ids) - max_length, stride):
            input_chunk = token_ids[i:i+max_length]
            target_chunk = token_ids[i+1:i+max_length+1]
            self.input_ids.append(torch.tensor(input_chunk))
            self.target_ids.append(torch.tensor(target_chunk))
        def __len__(self):
            return len(self.input_ids)
        def __getitem__(self, idx):
            return self.input_ids[idx], self.target_ids[idx]
```

With this class it is easy to create a data loader using PyTorch's DataLoader, that will finally load the inputs in batches and generate input-target pairs.

1. Text is converted to Tokens.
2. Tokens are organized into Samples (sliding window) using our dataset class.
3. Samples are grouped into Batches and randomized for the training loop.

We can test data loader by changing its parameters, specifically max length and stride.

```
dataloader = create_dataloader_v1(raw_text, batch_size=2, max_length=x,  
stride=y, shuffle = False)  
data_iter = iter(dataloader)  
first_batch = next(data_iter)
```

Test 1: max length and stride are both 2. (x = 2, y = 2)

Results in:

```
[ tensor([ [ 40, 367],  
           [2885, 1464]]),  
  tensor([ [ 367, 2885],  
           [1464, 1807]])]
```

This shows that the outputs are indeed the inputs shifted by 1, and because max_length = stride, the next inputs are direct neighbor tokens.

Test 2: max length is 8 and stride is 2. (x = 8, y = 2)

Results in:

```
[ tensor([ [ 40, 367, 2885, 1464, 1807, 3619, 402, 271],  
           [ 2885, 1464, 1807, 3619, 402, 271, 10899, 2138],  
           [ 1807, 3619, 402, 271, 10899, 2138, 257, 7026]]),  
  tensor([ [ 367, 2885, 1464, 1807, 3619, 402, 271, 10899],  
           [ 1464, 1807, 3619, 402, 271, 10899, 2138, 257],  
           [ 3619, 402, 271, 10899, 2138, 257, 7026, 15632]])]
```

By setting a stride of 2, the dataloader shifts the starting position of each sequence by only two tokens. As max length is 8, overlapping samples are created which allow the model to see the same data in different contextual positions.

2.4 IDs to Token Embeddings

The final step in text preparation for LLMs is to convert token IDs into embedding vectors. The manual process of mapping token IDs to embedding vectors is automatically implemented by the `torch.nn.Embedding` function. Further explanation is provided in Appendix D. The vectors provide smooth numeric representations of tokens so that language can be learned and processed by neural networks easily.

Embedding layers are initialized with random weights, and the random weights are learned during training through backpropagation. We need 2 parameters here:

1. **vocab_size** specifies the number of token IDs in the vocabulary, and therefore the number of embedding vectors that need to be initialized.
2. **output_dim** defines the dimensionality of each embedding vector. A higher value allows the model to capture more complex relationships between tokens, but also increases computational cost.

BPE has a vocabulary of size 50257 and if 256 dimensional embedding vectors are needed:

```
vocab_size = 50257
output_dim = 256
token_embedding_layer = torch.nn.Embedding(vocab_size, output_dim)
```

Now, we can use this weight matrix to map input IDs to embedding vectors. This is done easily, by mapping the ID to the embedding vector that is present in the weight matrix at the position corresponding to the token ID.

So, each token in the vocabulary has a random vector assigned to it and based on ID this vector becomes embedded vector of specific token. This implementation can be illustrated by Figure 2.4 with vocab size 6 and embedding dimension of 3.

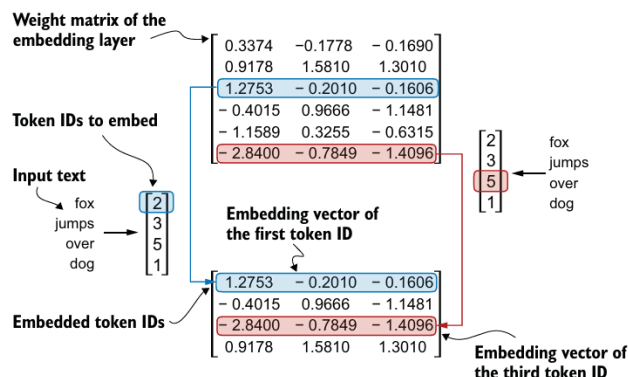


Fig. 2.4: lookup operation (Source: [1], p. 44)

2.5 Token Embeddings to Input Embeddings

Without additional information, the same token ID is always mapped to the same vector, regardless of its position in the sentence. To address this, **positional embeddings** are introduced.

Positional Embeddings can be generated in two main ways:

1. Absolute Positional Embeddings

- Assign a unique embedding to each position in a sequence.
- Added directly to token embeddings, ensuring each token carries both its identity and its location.

2. Relative Positional Embeddings

- Encode the distance between tokens rather than fixed positions.
- Enable the model to generalize better to sequences longer or shorter than those seen during training.

GPT-like models use absolute positional embeddings, so that is how it will be implemented in this project. First, generate positional embedding tensors, which have the same dimension as the token embeddings. We need as many positional embeddings as there are tokens in one input sequence, since we will not need more than that.

```
context_length = max_length
pos_embedding_layer = torch.nn.Embedding(context_length, output_dim)
pos_embeddings = pos_embedding_layer(torch.arange(context_length))
print(pos_embeddings)
```

```
tensor([[ 1.6459, -1.2114, -0.9353, ..., -0.0461, -0.9937, -0.0690],
        [ 1.4213, -2.6221, -2.5158, ..., -1.3071, -1.0946, -2.8479],
        [-0.0892,  0.5589,  0.5502, ...,  0.0323, -0.4755, -0.0566],
        [-1.3638, -0.0531, -0.4435, ...,  1.0636, -1.8907, -1.1526]],
        grad_fn=<EmbeddingBackward0>)
```

The final input for the LLM is generated by adding the token embeddings and positional embeddings.

```
input_embeddings = token_embeddings + pos_embeddings
print(input_embeddings)

tensor([[[ 0.0559,  0.1209,  0.8164, ..., -0.9750, -1.0423, -0.8083],
         [ 0.3718,  2.0368, -0.2498, ...,  1.8983, -3.7303, -0.5467],
         [-0.4809, -0.4057,  0.7816, ...,  0.6407,  3.1315,  0.4185],
         [ 0.8596,  0.8846,  1.4093, ..., -0.1684, -0.1808, -0.1241]],

        [[ 1.1107,  0.7339, -1.4272, ..., -0.6759,  1.6896, -1.0058],
         [-1.9384,  1.5918, -0.3655, ...,  2.5206, -2.8807,  1.2468],
         [ 1.7545, -0.9993, -0.0286, ..., -0.7052, -0.1777,  1.9191],
         [ 0.8787,  1.3162, -1.8305, ...,  1.4323,  2.1449, -0.0756]]],
        grad_fn=<AddBackward0>)
```

And now each token embedding is enriched with positional information to create the final **input embeddings** for the LLM.

3. Code Attention Mechanism

Before a formal explanation of an attention mechanism, we present a useful analogy: The attention mechanism can be compared to using a highlighter while reading. The reader does not pay equal attention to every single word on a page so they focus on the most important words to understand the main idea.

In the same way, an attention mechanism helps a large language model focus on the most relevant parts of an input sentence to better understand its meaning and context, and perform a task. Without attention, Neural Networks suffer from information bottlenecks. By trying to force an entire sequence into one hidden state, they frequently lose context in complex sentences with distant dependencies.

The attention mechanism addresses this by giving the model access to all parts of the original text at every step. As shown in figure 3.1, when the model is generating a new token, it can look back at the entire input, pay attention to the most relevant parts and decide which words are most important for predicting the next word in the output.

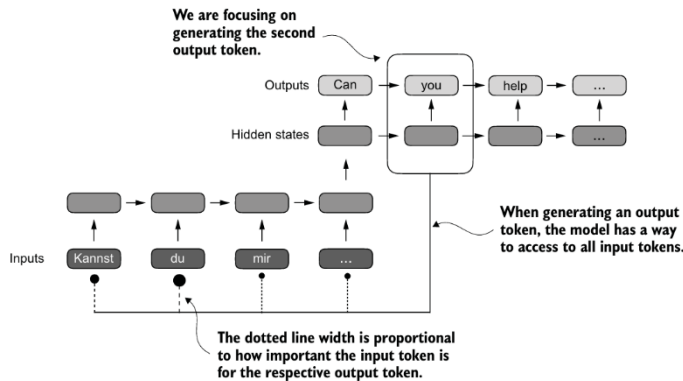


Fig. 3.1: attention mechanism
(Source: [1], p. 54)

As a result of the attention mechanism, the model can weigh the importance of every piece of incoming information to determine which parts are most relevant at any given moment.

3.1 Simplified self-attention without trainable weights

The simplified self-attention mechanism works by computing a context vector for every input token. Unlike a standard embedding, this context vector is enriched with information from the entire sequence, allowing the model to interpret a token's meaning based on its specific context rather than in isolation.

The Detailed Process:

First, we assume that each input token has a fixed-length embedding vector representation that encodes semantic information about the token (word) itself.

Step 1: Compute Attention Scores ω :

Let one input token be the current query, for which now we will calculate the context vector. We call its embedding vector q_i .

For a given query q_i , an attention score is calculated for each input vector separately and $\omega_{i,j}$ equals to the dot product of q_i and embedding vector of j^{th} input token: $\omega_{i,j} = q_i \cdot x_j$

The dot product approximates similarity between embeddings: higher values indicate stronger connection between the two tokens. The idea comes from the cosine similarity.

In practice, computing attention scores for all queries and all tokens results in an attention score matrix, which can be efficiently computed as a matrix multiplication:

$$\text{attn_scores} = \text{inputs} @ \text{inputs.T}$$

Step 2: Normalize via Softmax:

Step 1 produces the attention scores, which now should be converted into attention weights, so that they represent relative values of how close each token is to the current query. attention weight $\alpha_{i,j}$ is calculated using the softmax function: $\alpha_{i,j} = \text{softmax}(\omega_{i,j})$

Softmax normalizes the scores so that the weights for each query sum to 1.

Step 3: Calculate Context Vectors Z:

Step 2 produces attention weights between the query token q_i and all input tokens. After that, each input embedding x_j is multiplied by corresponding weight $\alpha_{i,j}$ and the resulting vectors from all the inputs are summed to produce the final context vector for q_i .

$$z_i = \sum_j \alpha_{i,j} x_j$$

This operation combines information from all tokens, however, due to larger weights, it gives more importance to the tokens most relevant to the query.

This process is repeated using each token in the sequence as the query. So finally, each token has its own context vector, which now reflects information about the importance of other inputs too. But we can improve it with trainable weights.

3.2 Self-attention with trainable weights

While the simplified self-attention mechanism demonstrates the core concept, its major limitation is that the attention scores are fixed, preventing the model from learning.

We need to make a strategy, so that instead of comparing raw input embeddings directly, the model learns how to look at the embeddings. As introduced in [4], this is achieved by projecting each input embedding into three different learned representations. You can see the theoretical details in Appendix E while we discuss practical step by step implementation here.

Step 1: Project inputs into Q, K, and V

Each input embedding is multiplied by the corresponding weight matrix:

```
queries = inputs @ W_query  
keys = inputs @ W_key  
values = inputs @ W_value
```

Step 2: Compute scaled attention scores

For each query token, attention scores are computed using the dot product with all keys:

$$\omega_{i,j} = q_i \cdot k_j$$

The scores are scaled by $\sqrt{d_k}$ to prevent large values that would destabilize training:

```
attn_scores = queries @ keys.T  
attn_scores_scaled = attn_scores / d_k**0.5
```

Step 3: Normalize to attention weights

The scaled scores are converted into attention weights using softmax. Each row of the resulting matrix sums to 1 and represents the relative importance of all tokens with respect to the current query.

Step 4: Compute context vectors

The context vector for a query token is computed as a weighted sum of value vectors:

$$z_i = \sum_j \alpha_{i,j} v_j$$

This operation integrates information from all tokens, emphasizing those most relevant to the query.

We can see that even though Self-attention with trainable weights ([sa_v2](#)) uses a different weight initialization scheme than Simplified self-attention ([sa_v1](#)), both implementations are otherwise similar. We can transfer the weight matrices from a one class object to another, such that both objects then produce the same results:

```
sa_v1.W_query = torch.nn.Parameter(sa_v2.W_query.weight.T)  
sa_v1.W_key = torch.nn.Parameter(sa_v2.W_key.weight.T)  
sa_v1.W_value = torch.nn.Parameter(sa_v2.W_value.weight.T)
```

With these trainable weights, the self-attention mechanism can now learn context-dependent relationships from data, forming the basis for the more advanced attention structures used in LLMs.

3.3 Causal attention that hides future words

For language generation tasks, a model must predict the next word based only on the words that have come before it. Standard self-attention allows every token to see every other token in the sequence, so the model can simply copy the next word instead of learning to predict it.

Causal attention (also known as masked attention) ensures that when calculating the context vector for a given token, the model can only attend to the current token and all previous tokens in the sequence. As shown in Figure 3.2, this is achieved by applying a mask to the attention weights directly or to the attention scores before the softmax function.

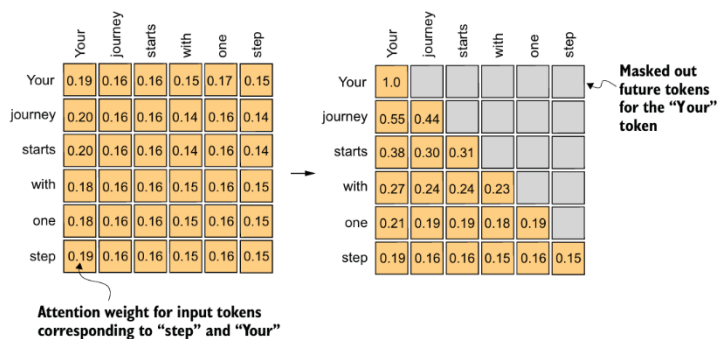
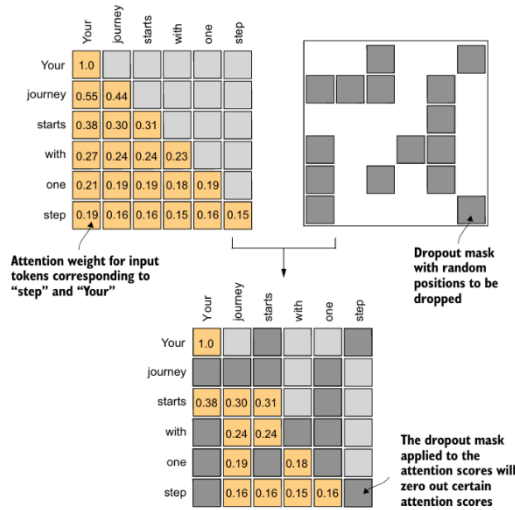


Fig. 3.2: causal attention (Source: [1], p. 74)

The most efficient way of implementing the causal mask is replacing the attention scores of all future positions with $-\infty$, that will result in 0 after the softmax function, as $\lim_{x \rightarrow -\infty} e^x = 0$. The process builds directly on the previous self-attention mechanism.

We also apply dropout to the attention weights to reduce overfitting during training.



Regularization/Dropout acts as a training filter. By randomly nullifying attention weights, it prevents the model from relying too much on specific words. Such random dropouts force the network to learn multiple paths for the same information, ensuring less dependency on training data and more generalization.

Fig. 3.3: causal attention with dropouts (Source: [1], p. 79)

3.4 Multi-head attention

The causal self-attention mechanism is powerful but it can only learn one type of relationship between tokens per layer. This is further improved by multi-head attention, which runs the attention mechanism several times in parallel, each time with its own set of learned weight matrices. It Allows the model to simultaneously focus on different parts of the input. As one head might discover syntactic relationships and another learn semantics relationships.

The outputs from these parallel heads are then combined, creating a final output with a more complete understanding of the text.

Conceptual Implementation: Stacking Attention Heads

The straightforward way to implement multi-head attention is to run multiple CausalAttention blocks with their own weights and then combine the output of each head. However, the simpler it is to implement, the less efficient the process is, since it performs multiple separate computations.

Efficient Implementation: The Standard Approach

In order to avoid separate runs, the good strategy is to perform one large matrix multiplication for all heads and then split the output. This involves the following steps:

1. **Projection to a large matrix:** Transform the input data into one large set of queries (Q), keys (K), and values (V) for all attention heads together. The total output size (d_{out}) must be divisible by the number of heads (num_heads).
2. **Split K,Q,V into individual heads:** Divide the large Q, K, and V matrices into smaller heads. Each head receives its own portion of the data so it can focus on different patterns or relationships. A tensor of shape $(batch, num_tokens, d_{out})$ is reshaped into $(batch, num_heads, num_tokens, head_dim)$, where $head_dim = d_{out} / num_heads$.
3. **Attention Computation for each head parallelly:** All heads calculate attention at the same time. This allows the model to process information much more efficiently.
4. **Concatenate Heads' outputs into a single matrix:** After each head finishes its work, join their outputs back together into one larger matrix.

5. **Output Projection:** Pass the combined output through a final linear layer `out_proj` to mix the information from all heads and produce the final model output.

Encapsulation in a Final MultiHeadAttention Class

The Multi-Head Attention class unites everything we discussed so far.

- Each word first creates three versions of itself: what it is looking for (query), what it represents (key), and what information it carries (value). Using these, the word decides which earlier words matter most to it.
- This process does not happen just once. It happens multiple times in parallel, in different “heads”. Each head looks at the sentence in a slightly different way, so the model can notice different patterns at the same time.
- A mask makes sure a word cannot see the future, only itself and what came before. The important words are weighted more, their information is combined, and each word gets an updated, context-aware representation.
- At the end, all heads are merged back together into one final vector per word.

Using the MultiHeadAttention class, we can initialize a multi-head attention module that has 12 attention heads, input and output embedding sizes of 768 dimensions and context length of 1,024 tokens, with 0 dropout rate. This exactly matches the configuration of the smallest GPT-2 model:

```
context_length = 1024
d_in, d_out = 768, 768
num_heads = 12
mha = MultiHeadAttention(d_in, d_out, context_length, 0.0, num_heads)
```

This efficient MultiHeadAttention module is the complete, production-ready version that will serve as a foundational building block for the full LLM architecture.

4. Implement a GPT Model

In this section, we present the implementation of a GPT model developed from scratch, including all architectural components required before the training phase.

Before starting the implementation, it is essential to understand the fundamental building blocks that create the model. Figure 4.1 provides a high-level overview of the GPT architecture that will be implemented and analyzed throughout this section.

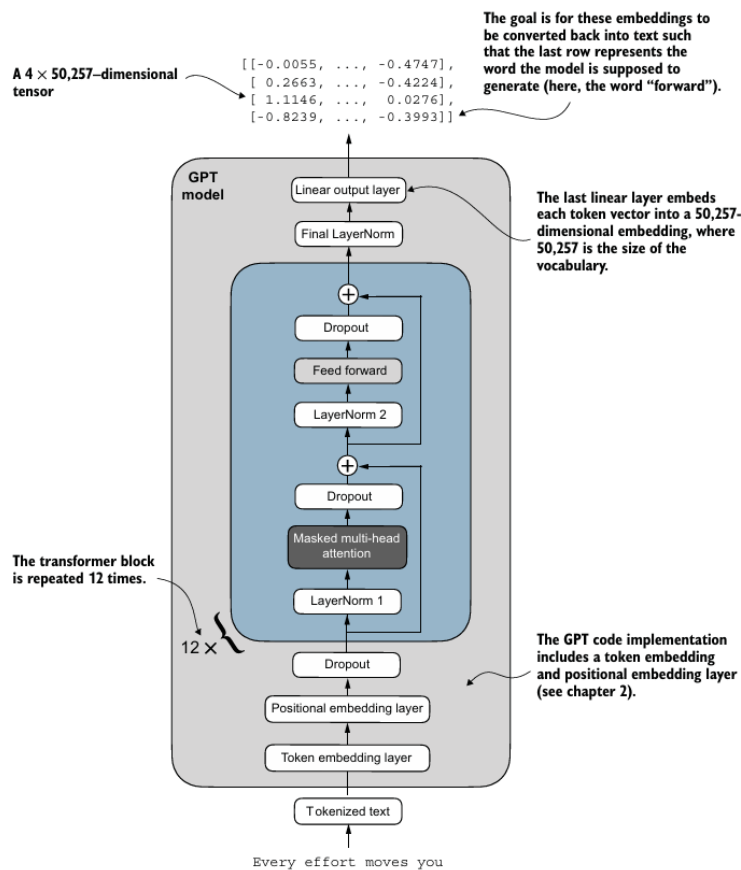


Fig. 4.1: An overview of the GPT model architecture (Source: [1], p. 118)

At a high level, a GPT model processes an input sequence of tokens through several stages:

- First, **token embeddings** convert token indices into continuous vector representations that are combined with positional embeddings so the model also considers word order.
- These representations are then passed through a sequence of **transformer blocks** all sharing the same internal structure and two main subcomponents:
 - **Masked multi-head self-attention mechanism**
 - **Position-wise feed-forward network**

Both topics are further explained in this section.

- **Shortcut connections, layer normalization, and dropout** are applied throughout the model to stabilize training and improve generalization.
- After passing all transformer blocks, the final representations are projected through a **linear output layer**, producing one score for every possible word in the vocabulary (logits), representing the model's next token prediction.

GPT-2 Small model (124M parameters):

```
GPT_CONFIG_124M = {  
    "vocab_size": 50257,      # Vocabulary size  
    "context_length": 1024,  # Context length  
    "emb_dim": 768,          # Embedding dimension  
    "n_heads": 12,           # Number of attention heads  
    "n_layers": 12,          # Number of layers  
    "drop_rate_attn": 0.1,   # Dropout rate for multi-head attention  
    "drop_rate_shortcut": 0.1, # Dropout rate for shortcut connections  
    "drop_rate_emb": 0.1,    # Dropout rate for embedding layer  
    "qkv_bias": False        # Query-Key-Value bias  
}
```

Instead of using a single global dropout rate, we defined separate dropout parameters for different parts of the model: the embedding layer, the shortcut connections, and the multi-head attention module.

- **drop_rate_emb** is used in the embedding layer
- **drop_rate_shortcut** is used in the residual (shortcut) connections inside each Transformer block (explained later)
- **drop_rate_attn** is input to the multi-head attention module

This decoupling enables more granular regularization control, leading to greater flexibility and potentially better model performance.

4.1 Layer Normalization

Training deep neural networks like GPT can be challenging because, when information passes through many layers, the signal used to learn can become weaker and weaker. If the signal is too small, it is hard for the model to learn because it does not know how to adjust its weights correctly. This problem is addressed by **layer normalization** that maintains the values within each layer well-balanced, resulting in more stable learning and allowing the model for efficient training.

The core idea of layer normalization is to normalize the output of a neural network layer so that the values for each individual input have **a mean of zero and a variance of one**.

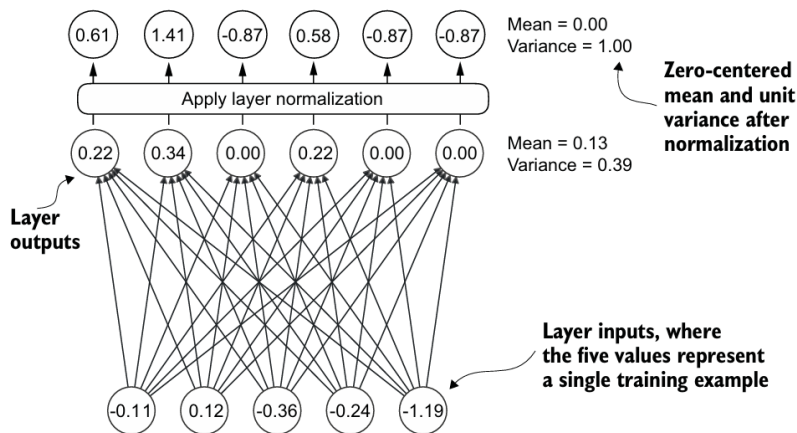


Fig. 4.2: Layer normalization
(Source: [1], p. 100)

Figure 4.2 illustrates our goal. Normalization can be achieved by subtracting the mean from the input vector and dividing the result by the square root of the variance augmented with a small constant ϵ , which is included to prevent division by zero.

$$\text{LayerNorm}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

In GPT and other transformer models, layer normalization is applied at several points in each transformer block. The normalization is performed over the embedding values of each token, and the result is then adjusted using learnable **scaling** and **shifting** parameters so the model can choose the most useful value range. Because layer normalization works independently for each input and does not rely on batch statistics, it remains stable even with varying batch sizes, making it a key component for training large language models reliably.

4.2 Activation Functions

Activation functions add nonlinearity to neural networks, enabling them to model non-trivial patterns beyond simple linear transformations. In transformer-based architectures like GPT, the **Gaussian Error Linear Unit (GELU)** is often used.

The GELU function is defined as $GELU(x) = x \cdot \Phi(x)$

where $\Phi(x)$ is the cumulative distribution function of standard Gaussian distribution.

In practice, GPT-style models use the approximation that is more computationally efficient and still closely matches the exact GELU:

$$GELU(x) \approx 0.5 \cdot x \cdot \left(1 + \tanh \left[\sqrt{\frac{2}{\pi}} \cdot (x + 0.044715 \cdot x^3) \right] \right)$$

To see an advantage of GELU over famous RELU function, please check out appendix F.

4.3 Feed-forward network

Besides the self-attention, each transformer block has a feed-forward network that processes each token representation independently. While attention combines information across tokens, the feed-forward network performs non-linear transformations, refines and enriches these representations and improves the model's ability to capture complex patterns.

In GPT, the feed-forward network is implemented as a small neural network with **two linear layers and a GELU activation function in between**. The first linear layer expands the embedding dimension by a factor of four, which allows the model to explore a richer set of feature interactions. Then the GELU activation introduces nonlinearity. Finally, the second linear layer projects the representation back to the original embedding dimension.

By keeping the input and output dimensions the same, transformer blocks can be stacked. This allows the feed-forward network to increase the model's ability to learn complex patterns while preserving a simple and flexible structure.

4.4 How Shortcut Connections Help

Shortcut connections are used in deep neural networks to address the vanishing gradient problem. Instead of forcing information to pass only through a long sequence of transformations, shortcut connections add a layer's input directly to its output. This provides an easier path for both information and learning signals to flow through the network.

To illustrate the effect of shortcut connections, we construct a deep neural network composed of multiple linear layers followed by GELU activations and compare two versions of the model: one with shortcut connections and one without.

Layer	Model Without Shortcuts	Model With Shortcuts
Layer 5 (Output)	0.05600	0.04165
Layer 4	0.01502	0.00583
Layer 3	0.00622	0.00197
Layer 2	0.00049	0.00027
Layer 1	0.00028	0.00271
Layer 0 (Input)	0.00011	0.00278

It is clear that in the model without shortcut the gradients become smaller as it progresses from the last layer (layers.5) to the first layer (layers.0), which is a phenomenon called the **vanishing gradient problem**. When gradients vanish, earlier layers receive very little learning signal, making it difficult for the network to update its parameters effectively.

In contrast, Model with shortcut demonstrates the gradient value stabilizes as we progress toward the first layer and does not shrink to a vanishingly small value. Therefore, shortcut connections enable effective learning across the entire network.

4.5 Transformer Block

The **transformer block** is the fundamental building unit of modern large language models. It is instantiated twelve times in GPT-2 Small architecture and comprises a number of building blocks we discussed earlier such as **masked multi-head attention, feed-forward networks, layer normalization, GELU activation, dropout, and shortcut connections**. All of these operations maintain the fixed dimension of the embeddings (768) and thus make the stacking of multiple transformer blocks possible without affecting the shape of the inputs.

The transformer block utilizes a **Pre-Layer Normalization** design, meaning layer normalization is done before both the attention and feed-forward sub-layers.

The forward pass consists of two main stages:

1. **Self-Attention Stage:** The input is normalized, passed through the masked multi-head attention module, regularized via dropout, and added back to the original input via a shortcut connection.
2. **Feed-Forward Stage:** The result is normalized and passed through the FFN (applying nonlinear transformations to each token independently). After dropout, a second shortcut adds the input back to the output.

To compare the computational cost of transformer components, we examine the number of parameters in the feed-forward and multi-head attention modules:

```
tfb = TransformerBlock(GPT_CONFIG_124M)
total_params_ff = sum(p.numel() for p in tfb.ff.parameters())
total_params_att = sum(p.numel() for p in tfb.att.parameters())
```

The resulting parameter counts are:

- **Feed-forward module:** 4,722,432 parameters
- **Multi-head attention module:** 2,360,064 parameters

Thus, the feed-forward network contains approximately twice as many parameters as the attention module. This difference exists because, as shown before, the feed-forward network temporarily makes each token representation much larger before shrinking it back to its original size.

4.6 Develop GPT Model and assess different configurations

At this point we implemented and analyzed the individual building blocks of a GPT architecture. We now put together these components to produce a fully functional LLM model. The full GPT model is implemented as follows:

```
class GPTModel(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.tok_emb = nn.Embedding(cfg["vocab_size"], cfg["emb_dim"])
        self.pos_emb = nn.Embedding(cfg["context_length"], cfg["emb_dim"])
        self.drop_emb = nn.Dropout(cfg["drop_rate_emb"])
        # Stacked transformer layers
        self.trf_blocks = nn.Sequential(
            *[TransformerBlock(cfg) for _ in range(cfg["n_layers"])]
        )
        self.final_norm = LayerNorm(cfg["emb_dim"])
        # Project hidden states back to vocabulary size
        self.out_head = nn.Linear(
            cfg["emb_dim"], cfg["vocab_size"], bias=False)
    def forward(self, in_idx):
        batch_size, seq_len = in_idx.shape
        tok_embeds = self.tok_emb(in_idx)
        pos_embeds = self.pos_emb(
            torch.arange(seq_len, device=in_idx.device))
        x = tok_embeds + pos_embeds
        x = self.drop_emb(x)
        x = self.trf_blocks(x) # Core processing
        x = self.final_norm(x)
        logits = self.out_head(x) # Output unnormalized probabilities
        return logits
```

First, the forward pass maps token indices into embeddings, adds positional information, and feeds the resulting tensor to the stacked transformer blocks. After final normalization, the output is projected into the vocabulary space, producing logits that represent unnormalized

probabilities for the next token in the sequence. For an input of shape [batch_size, num_tokens], the output has shape [batch_size, num_tokens, vocab_size].

Assessing Different GPT Configurations

Using the same GPTModel class and modifying only the configuration dictionary with values shown in the table below, we instantiate four GPT-2 variants. This demonstrates the scalability of the architecture:

Model of GPT-2	Embedding Dimension	Number of Layers	Number of Attention Heads	Trainable Parameters (with weight tying)	Total Parameters	Approx. Size
Small	768	12	12	124,412,160	163,009,536	621.83 MB
Medium	1024	24	16	354,749,440	406,212,608	1549.58 MB
Large	1280	36	20	773,891,840	838,220,800	3197.56 MB
XL	1600	48	25	1,557,380,800	1,637,792,000	6247.68 MB

The results show that increasing the embedding dimension, number of attention heads, and number of transformer layers results in higher parameter count and memory usage, therefore requiring more computational and storage resources.

4.7 Testing the GPT Model and Generating Text

After implementing the GPT architecture, we do basic tests to confirm that the model correctly processes tokenized inputs, produces outputs with expected shape, and supports auto-regressive text generation.

We first encode two short input sentences using the GPT-2 tokenizer to construct a batch of token IDs:

```
tokenizer = tiktoken.get_encoding("gpt2")
batch = []
txt1 = "The Education is important"
txt2 = "People together can do"
batch.append(torch.tensor(tokenizer.encode(txt1)))
```

```
batch.append(torch.tensor(tokenizer.encode(txt2)))  
batch = torch.stack(batch, dim=0)
```

The resulting batch has shape [2, 4], which is two input sequences with four tokens each.

The shape of the output tensor of the GPT model when you pass it through is:

```
torch.manual_seed(123)  
model = GPTModel(GPT_CONFIG_124M)  
out = model(batch)  
print("\nOutput shape:", out.shape)  
RESULT: Output shape: torch.Size([2, 4, 50257])
```

Each vector in the last dimension contains the unnormalized scores (logits) of the model for every possible next tokens in the vocabulary.

Text Generation

When generating text, the model generates one token at a time, based on previous tokens.

This is called **autoregressive generation**. At each step:

1. Input to the model is the current sequence.
2. It predicts the scores (logits) for all the possible next tokens.
3. The softmax function takes these scores and converts them into probabilities.
4. Argmax function is used to get the most likely token and add it to the sequence.

This method is called **greedy decoding** as it always chooses the most likely next token.

```
def generate_text_simple(model, idx, max_new_tokens, context_size):  
    for _ in range(max_new_tokens):  
        # Crop current context if it exceeds the model's maximum supported-length  
        idx_cond = idx[:, -context_size:]  
        with torch.no_grad():  
            logits = model(idx_cond)  
        # Focus only on the last time step (the next token to be predicted)  
        logits = logits[:, -1, :]  
        probas = torch.softmax(logits, dim=-1)  
        #Greedy decoding  
        idx_next = torch.argmax(probas, dim=-1, keepdim=True)
```

```
idx = torch.cat((idx, idx_next), dim=1)
return idx
```

After applying softmax, we get a vector of size 50257. Each value represents the probability of a token with index i , where $i \in [0, 50256]$

We use greedy decoding, therefore we take the index with the highest probability. That is what the **argmax** function returns.

This process is done for a fixed number of steps, gradually increasing the length of the input sequence and generating new tokens one at a time. The generated token IDs are then converted back to human-readable text using the tokenizer's decode method.

Generation Results and Interpretation

Now we apply the generation function to the sample inputs:

```
model.eval()
out = generate_text_simple(model=model, idx = batch, max_new_tokens = 7,
context_size=GPT_CONFIG_124M["context_length"])
for idx in out:
    decoded_text = tokenizer.decode(idx.tolist())
    print(decoded_text)
```

RESULT:

```
The Education is important criminal?) curriculumhern ic fres Ic
People together can doftime intensive💎 REALLY tumble Nay habitats
```

While the generated text is syntactically fragmented and largely incoherent, this behavior is expected. The model has **not been trained yet**.

These tests demonstrate that the implemented GPT model:

- correctly processes tokenized input,
- produces outputs with the expected dimensionality.
- supports iterative next-token generation.

Meaningful and coherent text generation requires training the model on large text corpora, which will be addressed in the next section.

5. Pretrain on Unlabeled Data

The goal of this section is to pretrain the model on unlabeled (raw) text data, that will allow our model to learn patterns of language. Unlike supervised learning tasks, language models do not require labeled datasets. They are trained to predict the next token in a sequence. By making this prediction over and over using big amounts of text, the model starts to learn grammar, semantic relationships, and contextual dependencies between words. However, before training the model, we need a performance measurement to check how well the model works.

5.1 Evaluating Generative Text Models

In the case of language models, performance means accuracy of the model in predicting the next token in a sequence. When the model assigns a high probability to the correct token, the prediction is good. If the probability is small, the prediction is poor.

The difference between the probability of token predicted by model and the actual token is called a **loss**. Smaller loss means better predictions. Please refer to Appendix G for the loss calculation algorithm.

During training the model tries to minimize this loss by tuning the parameters, which are initially generated randomly (or imported from an existing model). Hence, the loss value serves as an indicator of training progress, and a well-behaving model should show decreasing loss values over time.

To see if the model is learning general language patterns rather than memorizing the data, we evaluate the performance on 2 datasets:

- **Training set** – used to update model parameters
- **Validation set** – used only for evaluation

If the training loss decreases, but the validation loss is high, the model is likely **overfitting**, memorizes the training data instead of learning general patterns.

In language modeling, the correct output for each token is just the next token in the sequence. Hence, the training data is structured in a way that each input sequence is associated with a target sequence shifted by one position.

Example:

Input sequence	Target sequence
every effort moves	effort moves you

This means that the model learns to predict:

- given “every”, predict “effort”
- given “every effort”, predict “moves”
- given “every effort moves”, predict “you”

When repeating this task using large datasets, the model gradually learns patterns of language.

Role of Loss in Training

The loss value is central to training neural networks, because it tells us how wrong the predictions are. After the loss is calculated, the model performs **backpropagation**. This way it figures out how much each parameter in the network contributed to this error.

Backpropagation starts from the output layer and the algorithm calculates the change of the loss with respect to each of parameters. This gives a set of values called **gradients**. A gradient shows how to change a parameter to reduce the loss. If the gradient is large, the parameter has a strong impact on the error, otherwise, the parameter has little effect.

The optimizer takes these gradients and updates the model’s parameters so that the loss is minimized. Repeating this process for several training steps makes the model better and better at prediction.

The training process can be summarized in 3 steps:

Forward pass – the model makes predictions.

Loss calculation – the error between predictions and the correct values is measured.

Backpropagation – the model determines which parameters caused the error and how they should be adjusted.

5.2 Training and Validation Losses

We illustrate the training process on a small dataset containing the short story “The Verdict” by Edith Wharton, so that the process can be done in a manageable setting, but also be enough to show the mechanics of language model training.

1. Preparing the Dataset: The dataset is a raw text, which is converted into tokens using the tokenizer introduced in Section 2. The tokenized text is split into sequences matching the context length of the model (256 tokens in this implementation).

These sequences are grouped in batches, so the model can process multiple examples at the same time. The dataset is also divided into 90% training and 10% validation data to keep some part of it unseen during training.

2. Process the dataset: before these batches are given to the model, they are processed, so that each contains two tensors: one with input tokens and another with target tokens. This is done by **data loaders**. The target tokens are the input tokens shifted by one, so the model always learns to predict the next token in the sequence.

3. Computing Loss for the Dataset: For the overall performance estimation the loss is computed by averaging the loss values across all batches in the dataset. The final **dataset loss** is the accuracy of the model in predicting tokens over the entire dataset.

Normally, the training and validation losses are both high before training starts, because the model parameters are randomly initialized. The goal is that with training these values should decrease.

4. Interpreting Training and Validation Loss: Both loss values tell us something about the model’s learning behavior. During successful training:

- **training loss decreases** means that the model is learning patterns in the data
- **validation loss also decreases**, so the model generalizes to unseen text

5.3 LLM Training Loop Implementation

After implementing the training and validation loss calculation, the next step is to train the language model. Training tunes the internal parameters to gradually improve its predictions and minimize the loss function.

The training loop is a standard procedure in deep learning and typically includes the steps shown in Figure 5.1.

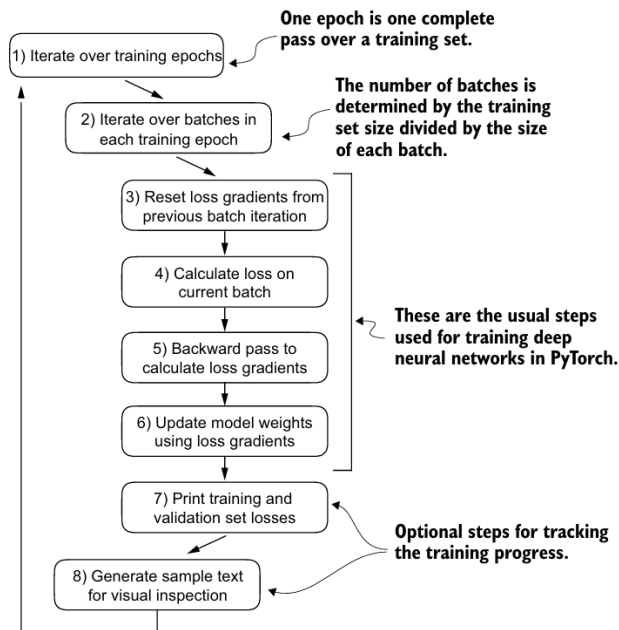


Fig. 5.1: A typical training loop (Source: [1], p. 146)

An **epoch** is one pass through the training data. Since datasets are generally large, it needs to be split into smaller batches so that it can be processed by the model.

At each iteration, the model receives a batch of token sequences and predicts the next token for each position. The predictions are then compared with the target tokens to compute the loss.

After it knows the loss, the model **backpropagates**, which results a set of gradients indicating how to change the parameters to decrease the loss. An optimization algorithm then uses these gradients to update the model weights. Over many iterations, these updates improve the model's ability to predict the next token.

For this implementation, we used AdamW optimizer, which is common standard for training transformer models. For more details about AdamW optimizer read Appendix H.

Training Results

While training, it is important to monitor the model's progress to ensure that learning occurs as expected. Training loss measures performance on the training data, while validation loss measures performance on unseen data.

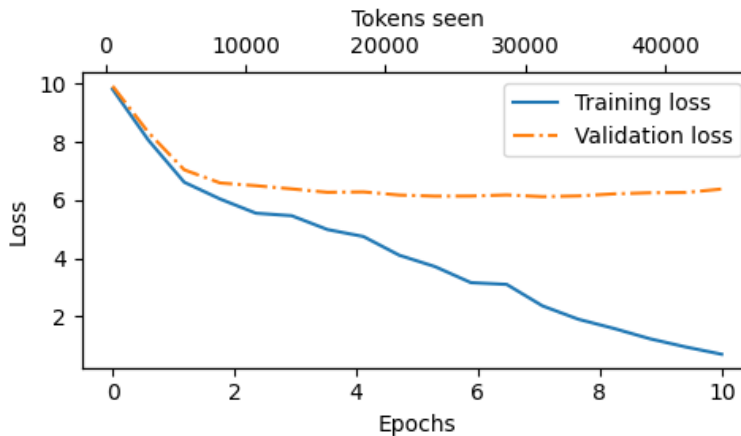


Fig.5.2:
Simple Training Results

As we see in Figure 5.2, in the beginning of training both losses are very high and almost equal. Because the model weights are randomly initialized, it assigns almost same probabilities to all tokens.

$$p \approx \frac{1}{50257} \quad \rightarrow \quad -\log\left(\frac{1}{50257}\right) \approx 10$$

Our loss ≈ 9.9 is exactly what we expect.

In the early learning phase training loss is dropping quickly, as the model is learning basic patterns: common words, punctuation, simple token relationships. Even simple improvements drastically reduce loss.

In general Training loss keeps dropping strongly. Since the dataset is small, the model can almost remember the exact sequences. On the other hand, validation loss stops improving and even gets slightly worse. This is overfitting. However, this behavior is expected because the dataset contains only a few thousand tokens while the model has a much larger capacity.

In addition to numerical metrics, it is also useful to periodically generate text samples during training.

Epoch	Generated text
1	Every effort moves you,,,,,,,,,,,,.
5	Every effort moves you know the "Oh, and he had to me--I me. "Oh, I felt--and it's the "Oh, and I had been the donkey--and it to me, and down the "Oh,
10	Every effort moves you?" "Yes--quite insensible to the irony. She wanted him vindicated--and by me!" He laughed again, and threw back his head to look up at the sketch of the donkey. "There were days when I

At epoch 1, the model produces grammatically broken output, indicating no learned structure yet. By epoch 5, punctuation and short phrases emerge, but the model fixates on patterns like 'Oh,' suggesting it is latching onto frequent tokens. By epoch 10, the output is almost same as the actual text in *The Verdict*. This is a clear sign of memorization rather than generalization, already observed in Figure 5.2 with the diverging training/validation losses.

However, if the training set is small, the model might start to memorize the training data. In those situations, generated text may repeat passages from the dataset rather than generating new content.

That is why today's language models are typically trained on very large datasets that contain billions of tokens.

5.4 Improving the Training Loop

The training loop implemented earlier provides a simple example of how a language model can be pretrained. However, modern large language model training typically involves additional techniques that help improve training stability and performance.

Learning rate warmup

Training can be unstable at the beginning if the learning rate is too high. This is solved by Learning rate warmup where it starts with a small learning rate and increases it gradually up to the peak value.

```
lr_increment = (peak_lr - initial_lr) / warmup_steps
if global_step < warmup_steps:
    lr = initial_lr + global_step * lr_increment
else:
    lr = peak_lr
```

This allows the model to train carefully at the beginning, and avoids unstable updates at the early stage. Warmup usually takes up 0.1% to 20% of the total training steps.

Cosine learning rate decay

After the warmup phase, the learning rate is decreased gradually via cosine decay. This helps to make larger updates early in training and smaller updates later, when it approaches a good solution. The learning rate can be computed as:

```
progress = ((global_step - warmup_steps) /
            (total_training_steps - warmup_steps))
lr = min_lr + (peak_lr - min_lr) * 0.5 * (1 + math.cos(math.pi * progress))
```

where *progress* indicates how far training has advanced.

This results in a smooth schedule where the learning rate first increases and then decreases smoothly (See figure 5.3). Cosine decay helps the model converge smoother.

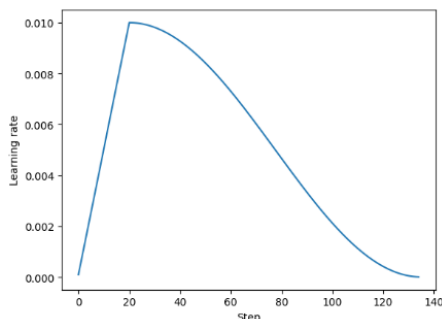


Fig. 5.3: warmup followed by a cosine decay

Gradient Clipping

Training deep networks can also suffer from exploding gradients, where gradients become excessively large. Gradient clipping limits their magnitude to stabilize training.

```
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

If the gradient norm exceeds the threshold, the gradients are scaled down so that their magnitude stays within the specified limit. This stabilizes training, especially for deep transformer models.

Modified Training Function and Results

The training loop's overall structure is unchanged, but the learning rate is modified dynamically during training. These improvements are commonly used in modern transformer training pipelines and help achieve more stable and efficient optimization.

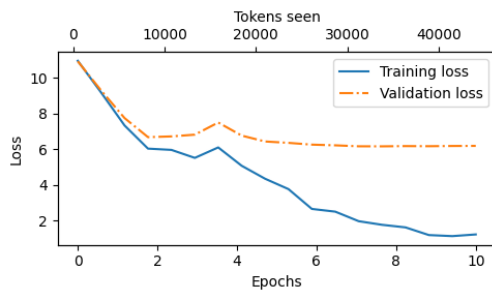


Fig. 5.4: Improved Training Results

Due to hardware constraints, the model was trained on a relatively small dataset. As a result, overfitting occurs even when using the improved training process (See Figure 5.4). Although the overall results are similar to the previous training, the training curves differ.

5.5 Decoding Strategies (temperature, top-k)

After training a language model, we can already use it to generate text. However, the quality and originality of the obtained text depend not only on the model itself but also on decision of which token will be the next. The method used to choose the next token is called a **decoding strategy**.

The most straightforward way is Greedy Decoding, which simply selects the token with highest probability at each step. It gives more coherent output, but it can also be repetitive or predictable. To fix this and introduce variety with randomness, **temperature scaling** and **top-k sampling** are the most used techniques. They balance coherence and diversity during text generation.

Temperature Scaling

Temperature scaling changes the probability distribution before selecting the next token. It divides model's logits by a temperature and only after applies the softmax function. The temperature value will determine how sharp or spread out the probability distribution will be:

- **temperature = 1** → original distribution
- **temperature < 1** → sharper distribution (more deterministic output)
- **temperature > 1** → flatter distribution (more diverse output)

Higher temperatures mean more creativity but may also produce grammatically incorrect or nonsensical sentences.

To study the influence of temperature, we computed the probability distribution for different temperature values: **0.01**, **0.1**, **1**, **4**, and **16**. For each temperature we generated the next token 1000 times and checked the frequency of each.

Temperature	closer	every	effort	forward	inches	moves	pizza	toward	you
0.01	0	0	0	1000	0	0	0	0	0
0.1	0	0	0	985	0	0	0	15	0
1	73	0	0	582	2	0	0	343	0
4	173	69	31	268	63	34	23	251	88
16	128	109	85	163	83	93	80	148	111

To interpret the results, we noted down that the most suitable next token would be “forward” and the most irrelevant – “pizza”. The results show that for very low temperatures (0.01 and 0.1), the probability distribution becomes so sharp that almost all probability is assigned to the most likely token (“forward”). However, as the temperature rises the most mismatched token gets more and more probability to be chosen (“pizza” reaching almost 8% at a temperature of 16).

This behavior explains why very high temperatures often produce more creative but less coherent text, while very low temperatures lead to deterministic outputs.

Top-k Sampling

Yet another popular decoding strategy is known as "top-k sampling". Rather than selecting from the whole vocabulary, the model samples the next token only from the k most probable ones, all other tokens' probability becomes 0.

This prevents extremely unlikely tokens from being selected while still allows some diversity in the output.

The impact of both temperature and the top-k was studied for several different combinations of the parameters in our local and GPT2 small models. Note that a top_k value of 1 and a temperature of 0 always produces completely deterministic output.

Model	temperature	top_k	Example generated output
Local	1.3	5	Every effort moves you know terrace. "I glanced after him, with a little last
Local	1.6	25	Every effort moves you as though, or Florence me afterward ins geniused that dep Mrs of such
Local	0	1	Every effort moves you?" "Yes--quite insensible to the irony. She wanted
GPT2-s	1.3	5	Every effort moves you forward in the life of the person, but it's a very hard thing
GPT2-s	1.6	25	Every effort moves you to reach where a particular value is most effective, and you try all these
GPT2-s	0	1	Every effort moves you forward. The first step is to understand the importance of your work

As expected, experiments also proved that **temperature** and **top-k** strongly affect the randomness and structure of the output.

With **top_k = 5**, the model selects from five most probable tokens, which makes more predictable and coherent sentences, helpful for tasks requiring accuracy. Examples can be code generation or multiple-choice question answering.

With **top_k** still **5**, but **temperature** increased to **1.3**, the local model generated incoherent text (*"know terrace"*) and stronger GPT-2 model generated coherent. So, 1.3 temperature was already too high for model trained on only 4,000 tokens.

Increasing **top_k** to **25**, the model chooses from a larger pool of tokens, so the output will have more variety. A higher temperature (e.g. **1.6**) makes less probable tokens appear more often. This results in more creative text, but it might be less structured or in the worst case incoherent, so we need to choose the parameters carefully. High **top_k** and temperature values are good for storytelling or brainstorming.

Finally, with **temp=0** and **top_k=1** text generation is fully deterministic and always selects the most likely token.

5.6 Saving and Loading Model Weights

Training large language models is computationally expensive, so instead of training it from scratch every time, trained models are usually stored and reused.

In PyTorch, model parameters are stored in the state dictionary (`state_dict`), which is a dictionary mapping layer names to their corresponding tensors. To save a model, simply saving this state dictionary (usually in a `.pth` file) is enough, and later it can be loaded into a model with the same architecture.

If you need to resume training, you should also save the optimizer state, because optimizers like Adam or AdamW keep track of additional information about past gradients, which allows you to continue training where it was stopped. So, when we want to continue training the model in a new session, we need to load the previously saved checkpoint with not only the model, but also optimizer states.

1. First, the checkpoint is loaded and the model architecture is re-initialized with the same configuration. Then, the `load_state_dict` function is used to restore the saved model parameters.

```
checkpoint = torch.load("model_and_optimizer.pth", map_location=device)
model_imported = GPTModel(GPT_CONFIG_124M)
model_imported.load_state_dict(checkpoint["model_state_dict"])
model_imported.to(device)
```

2. Next, the optimizer is recreated and its state is restored from the checkpoint.

```
optimizer_imported = torch.optim.AdamW(
    model_imported.parameters(), lr=5e-4, weight_decay=0.1)
optimizer_imported.load_state_dict(checkpoint["optimizer_state_dict"])
```

With the model and optimizer states restored, training can continue as usual. We use the function `train_model_simple` with `num_epochs=1` to train the model for one more epoch.

```
model_imported.train()
num_epochs = 1
train_losses, val_losses, tokens_seen = train_model_simple(
    model_imported, train_loader, val_loader, optimizer_imported, device,
    num_epochs=num_epochs, eval_freq=5, eval_iter=5,
    start_context="Every effort moves you", tokenizer=tokenizer
)
```

Recall that the initial training loss at training step 0 was 9.821 and validation loss was 9.931. Following the first training process, these values were reduced to 0.708 and 6.387, respectively. When continuing training from the saved checkpoint (instead of starting from scratch), the first step already obtained a training loss of 0.550 and a validation loss of 6.417, indicating that the previously learned weights were successfully restored.

That is a proof that we can save the model parameters and the optimizer state and resume training without losing what the model have learned so far.

5.7 Loading Pretrained GPT-2 Weights

Training large language models from scratch is computationally expensive, and thus pretrained models such as **GPT-2** are often reused instead of training new models from the beginning. Pretrained weights can be loaded and used directly for text generation or further adapted through fine-tuning.

OpenAI released several GPT-2 variants that differ mainly in the number of parameters. The downloaded parameters can be mapped to the corresponding layers of the implemented GPT model, including token embeddings, positional embeddings, attention projections, feed-forward layers, and normalization parameters.

To evaluate the pretrained GPT-2 model on *The Verdict* dataset, the training and validation losses were calculated using the `calc_loss_loader` function.

Training loss: 3.5319405794143677 Validation loss: 3.5318045616149902

The results mean that the pretrained GPT-2 model does a reasonably good job on The Verdict dataset. However, this text might have been included in the data used to pretrain GPT-2. If that is true, the results would not fully reflect the model's ability to generalize to unseen text.

To investigate the model further, another experiment was performed on a different dataset. This dataset consists of recent English translations of Georgian history notes, produced by Giorgi Gogsadze, a co-author of this report. Since these notes were created after GPT-2 was trained, they could not have been part of its pretraining data. The obtained loss for this dataset was: 2.895763317743937

The comparable loss values suggest that the pretrained GPT-2 model captures general language patterns rather than relying solely on memorized training data.

Furthermore, we explored the effect of model size on text generation, the **GPT-2 small model (124M parameters)** and the **GPT-2 XL model (1558M parameters)** were tested using the same prompts. Two example prompts were used:

Prompt 1: *“Every effort moves you”*

- **GPT-2 small:**

“Every effort moves you as far as the hand can go until the end of your turn unless something happens. This would remove you from a battle.”

- **GPT-2 XL:**

“Every effort moves you further to where you do not actually want to grow; you are still too much of an outsider. This has been demonstrated clearly.”

Prompt 2: *“The Capital of Georgia is”*

- **GPT-2 small:**

“The Capital of Georgia is the place the capital can build infrastructure like a rail network. It's a very popular destination right after Georgia's economic prosperity.”

- **GPT-2 XL:**

“The Capital of Georgia is the place the capital can use because it was founded within Georgian territory but does not belong to it.”

As we can see from the examples above, the bigger GPT-2 XL model produced more fluent and well-connected sentences, while smaller model generated less structured texts.

This behavior is expected. Bigger models have more parameters, therefore they can capture more complex patterns and relationships between words. However, it comes with a price that they also require much more computational power.

6. Fine-tune for classification

This section explains how a pretrained GPT language model was fine-tuned for supervised classification task. While chapter 6 of [1] covers classification with 2 classes and small dataset, we chose to create our own project with any number of classes and larger dataset, and modify the codes accordingly.

This project is about emotion classification of short text messages. Even though intended goal of GPT models is to predict the next token in a sequence, the objective here is to predict which class does a text belongs to: *positive*, *neutral* or *negative*.

6.1 Dataset Preparation

The quality and organization of a dataset play a huge role in any classification system, and our system is definitely not an exception. In the context of classification fine-tuning of a LLM, dataset preparation essentially means turning an unstructured collection of raw text documents into a well-formed, numerical and organized data that our model can actually process during training.

Dataset Description

The dataset used in this work includes **31,015** Twitter posts, each labeled with one of three sentiment categories: **positive**, **neutral** or **negative**.

	Label	Text
0	neutral	I`d have responded, if I were going
1	negative	Sooo SAD I will miss you here in San Diego!!!
2	negative	my boss is bullying me...
...	...	...
31013	positive	_sutra what is your next youtube video gonna b...
31014	positive	http://twitpic.com/4woj2 - omgssh ang cute n...

An important feature of the dataset is its class distribution, but initially it is **imbalanced**:

- Neutral: 12,548
- Positive: 9,685
- Negative: 8,782

Class imbalance is a problem, because it can introduce a bias in the learning process. For example, the model might think that predicting “neutral” for all inputs would be good idea as it is the most probable class in the dataset.

To not let this problem appear, all classes are randomly filtered to match the smallest class.

The resulting balanced dataset contains:

- Positive: 8,782
- Neutral: 8,782
- Negative: 8,782

This makes all classes contribute equally during training and ensures that the loss function does not favor any class. It is true that undersampling reduces dataset size, but this method is very effective to solve the imbalance problems in classification tasks.

Note that neural networks work on numerical data, so labels must be converted into integers: positive will be represented as 0, neutral as 1 and negative as 2.

Dataset Shuffling and Splitting

To make even further refinements and ensure that the model does not learn patterns based on how data is ordered, we can shuffle the data randomly and only after divide it into three subsets:

- **Training set (70%)** - Used to update model parameters
- **Validation set (10%)** - Used to monitor performance and guide training decisions
- **Test set (20%)** - Used for final, unbiased evaluation

This separation ensures that overfitting can be detected and the model is evaluated not only on training data, but also on unseen data.

6.2 Data Loader Construction

After preparing the dataset, the next step is to transform the data into a form that is the best for efficient training of the model. This involves tokenization, sequence normalization, and batching, all of which are handled through a custom dataset class and PyTorch data loaders.

Variable-Length Sequences and the need for Padding

In pretraining the model uses fixed-length text chunks. However, in the classification dataset there are variable-length inputs, which is problem as batching requires that all sequences in a batch have the same length.

There are two main ways to address this problem:

1. **Truncation to shortest sequence:** efficient, but leads to information loss
2. **Padding to longest sequence:** preserves information, but increases computational cost

In this work, the second approach was selected, as the maximum input length in the dataset is sufficiently small to justify the additional computational cost.

The maximum sequence length was determined from the training dataset and turned out to be 70 tokens, which reflects the natural length of Twitter posts and is significantly smaller than the model's maximum context size (1024 tokens).

After tokenization all sequences are padded to a fixed length using the special token, that does not carry semantic meaning and acts as a placeholder:

`<|endoftext|>` (token ID 50256)

So, a sequence shorter than the maximum length becomes:

$[x_1, x_2, x_3] \rightarrow [x_1, x_2, x_3, 50256, 50256, \dots]$

An alternative configuration could be to pad all sequences to the full model context length (1024 tokens). But, after experiments, this resulted in a worse performance (**65% accuracy**).

This can be explained by the fact that 70 tokens represent less than 7% of the context window and classification head operates on the final token, therefore there is a huge positional gap between real end of a tweet and the final token of the constructed input.

Thus, selecting an appropriate sequence length is critical for performance.

Custom Dataset Implementation

All above mentioned features has been united into a custom dataset class. Its responsibilities include:

1. **Loading data from CSV files**
2. **Filtering invalid entries** (removing missing or empty text)
3. **Tokenizing text**
4. **Determining sequence lengths**
5. **Applying truncation (if necessary)**
6. **Applying padding**

Each dataset item returns:

- input tensor → sequence of token IDs
- label → integer class index

Batch Construction with Data Loaders

PyTorch DataLoader objects are used to group data into batches, which enables efficient iteration by avoiding processing inputs one by one. In this project we decided to increase batch size to 32. This decision was motivated by the relatively small maximum input length (only 70 tokens) and the large dataset size (over 30k samples). Increasing the batch size to 32 resulted in more stable training and improved computational efficiency due to parallelism.

Each batch has the structure:

- Input tensor ([32,70])
- Target tensor ([32])

6.3 Model Adaptation for Classification

The model will be loaded with pretrained GPT model weights before we start fine-tuning. However, GPT is originally designed for language modeling, where the objective is to predict the next token in a sequence. To apply this model to a classification task, its architecture and training objective must be modified.

From Language Modeling to Classification

In its original form, the GPT model outputs a tensor of shape:

(batch size, sequence length, vocabulary size)

For GPT-2, the vocabulary size is 50,257. However, for classification tasks we want to assign a single class label to the entire input sequence. Thus, the model must be adapted to map a sequence of tokens to a **fixed number of classes**, which in our case is 3.

The key architectural modification is the replacement of the original output layer, the one which produces logits, with a **classification head**. So instead of mapping embedding dimension into 50,257 logits, it will map to only 3.

Then for a given input, the model will output $[z_0, z_1, z_2]$, where each value corresponds to a class and the one with highest value will be the predicted class.

A **classification head** is a small neural network layer added on top of a pretrained model to adapt it for a specific task, such as classification.

In the original GPT model, the final layer (called the *output layer*) is designed for language modeling, where each output corresponds to a token in the vocabulary.

- $768 \rightarrow 50,257$ (next-token prediction)

For classification tasks, this behavior is not needed. Instead, we replace this output layer with a classification head, which takes the same 768-dimensional representation, but outputs a small number of values equal to the number of classes.

- $768 \rightarrow n$ (class prediction, where n is the number of classes)

This modification allows the model to reuse its learned language representations while adapting it to a new task.

Freezing Model Parameters

For this work, we used GPT-2 Medium (355M parameters), which is large model with improved ability to capture nuanced patterns. After initializing the model with pretrained weights, we prepare it for further training, specifically classification fine-tuning. However, not all layers in the model need to be trained. The lower layers typically capture general linguistic patterns, which should be preserved, while the higher layers learn task-specific features.

Thus, we freeze lower layers, which:

- reduces computational cost
- preserves useful pretrained knowledge
- avoids overfitting

As a result, only a subset of the model is trained, specifically:

- Final transformer block
- Final layer normalization
- Classification head

An alternative approach involves unfreezing all parameters:

```
for param in model.parameters():  
    param.requires_grad = True
```

This resulted in slightly improved accuracy to approximately **80%**, but at the same time training time increased to approximately **4 hours**.

This tells us that the full fine-tuning maybe enables the model to better align with task-specific patterns, but it is very expensive computationally.

Selecting the Relevant Output Token

Notice that the model produces an output for each token in the input, so, an input of length 70 results in 70 output vectors. However, classification requires only a single prediction per whole input. We know that the attention mechanism masks future tokens and only the final token has access to the full input context, therefore, the output of the last token is better to be used as a classification result.

In order to try out how meaningful it is to choose the last output, we conducted an experiment in which the indexing was changed from `(slice(None), -1, slice(None))` to `(slice(None), 0, slice(None))`. So, the output corresponding to the first input token was used as a final result. This modification resulted in an accuracy of approximately 70%, which is lower than the performance obtained when using the last token. This outcome is expected, as the first token has access to less contextual information than the final one.

6.4 Implement Evaluation Metrics

After adapting the model architecture for classification, the next step is to implement classification loss and accuracy functions, so we can evaluate model performance during training and testing.

Accuracy function

After the forward pass, the model produces logits of the form:

$$\text{logits} = \text{model}(x)[:, -1, :]$$

As mentioned above, these logits correspond to the **last token output**. Finally, the output logit will look something like this: `[-3.6345, 1.1317, -0.0587]`.

To obtain a prediction from this tensor, the index of the maximum logit is selected using `argmax` function. After getting predictions for all inputs, accuracy measures the proportion of correctly predicted labels:

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}}$$

Cross-Entropy Loss

We are using same loss function calculating strategy, that we defined for pretraining:

$$\mathcal{L} = -\log(p_y) \quad p_y \text{ is the predicted probability of the true class.}$$

This means:

- If the model assigns high probability to the correct class → low loss
- If the model is uncertain → higher loss
- If the model is confidently wrong → very high loss

Thus, minimizing cross-entropy encourages correct predictions.

Implementation Details

Although the loss is defined using softmax probabilities, in practice the function:

```
torch.nn.functional.cross_entropy(logits, target_batch)
```

operates directly on logits and internally applies the softmax transformation. Therefore, it is not necessary to explicitly compute probabilities during training.

Similarly, class predictions can be obtained using the **argmax** function on logits, without applying softmax, since both yield the same result.

As in the pretraining stage, the model is optimized using the **AdamW optimizer**, which is widely used for transformer-based architectures due to its stability and effective handling of weight decay.

Baseline Performance Before Fine-Tuning

Before training, the model achieves:

- Training accuracy: 34.38%
- Validation accuracy: 35.94%
- Test accuracy: 35.31%

For a 3-class problem:

$$\text{Random accuracy} = \frac{1}{3} \approx 33.3\%$$

Thus, the pretrained model performs randomly. This shows that pretrained GPT models do not inherently perform classification and task-specific fine-tuning is necessary.

6.5 Model Fine-Tuning and Analysis

The training algorithm necessary for fine-tuning is basically same as it was for pretraining, explained deeply in the section 5. The modification is that instead of generating sample text during training, the model is evaluated using classification accuracy after each epoch. This transits training from a language modeling task to a classification task.

Early Training Phase (Epochs 1–2)

- Stabilization around **Train loss 1.096, Val loss 1.092**
- Low accuracy **Training accuracy: 40.62% | Validation accuracy: 31.25%**

The model initially gives equal probability to all classes, meaning it is random. This is expected because the dataset is too large and two epochs were not going be enough for meaningful learning. For comparison, fine-tuning smaller dataset in Chapter 6 of [1] already reached 70% after the first epoch, therefore smaller datasets indeed show faster improvements.

Learning Phase Transition (Epoch 3)

From epoch 3 onward:

- Training as well as validation loss decreases rapidly (from ~1.09 to ~0.85 and below)
- Accuracy increases significantly (~70%)

This is a transition point when the model's trainable layers already start to learn meaningful, classifier-specific patterns. Such delayed improvement is also typical when most layers are frozen, as in our case.

Late Training Phase (Epochs 4–5)

Ep 4 (Step 001950): Train loss 0.695, Val loss 0.690

...

Training accuracy: 76.25% | Validation accuracy: 75.00%

...

Ep 5 (Step 002550): Train loss 0.659, Val loss 0.614

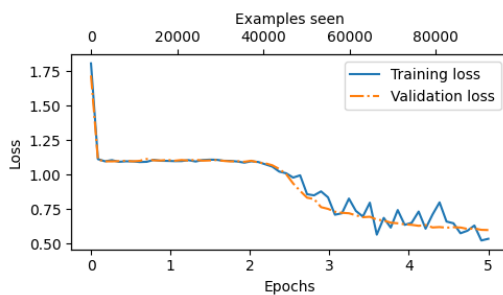
In final epochs:

- Both training and validation losses continue to decrease and remain close, indicating a good generalization and the absence of significant overfitting.
- Accuracy stabilizes at approximately 75%, suggesting convergence of the model.

Final Results

Training took 22.70 minutes, which is considered as very efficient, taking the dataset and model sizes into consideration. It also showed a solid result.

The loss plot:

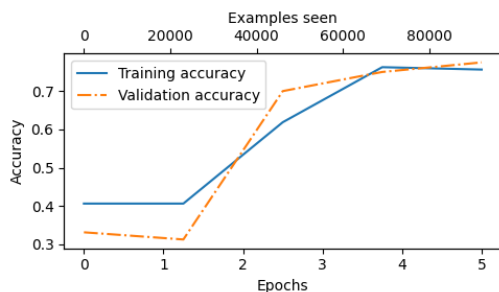


The sharp drop in the first few hundred examples is typical in classification fine-tuning, as classification head weights start shaping here. At this point guesses are purely random, and taking into consideration we have 3 classes, initial convergence to 1.1 is expected:

$$\mathcal{L} = -\log\left(\frac{1}{3}\right) \approx 1.098$$

However, in first and second epochs loss plateaus, staying close to the random-guessing baseline. Meaningful learning only begins in epoch 3 and by epoch 5 converges to 0.5. Overall, the curves look healthy: both accuracies rise together and plateau. No clear sign of overfitting.

The accuracy plot shows a very successful fine-tuning run for most practical purposes:



- Accuracy keeps improving until late epochs and then starting converging.
- Very slight divergence between train and validation metrics means almost no overfitting.
- validation metrics being slightly better than training metrics is a good sign of generalization. It is quite common in fine-tuning.

Final results showed:

Training accuracy: **76.04%**

Validation accuracy: **74.83%**

Test accuracy: **74.54%**

Overall, these numbers are consistent with each other with only a modest ~1.5% gap between training and test accuracy, meaning it generalized well.

Achieving **75%** accuracy on this task is reasonable, as the dataset was large, noisy with many ambiguous and subjective sentences. Perfect classification was indeed very challenging, but still the model shows strong capability of sentiment classification, which the qualitative result also verifies.

Qualitative Evaluation

In addition to quantitative metrics, a qualitative analysis was conducted by testing the model on several unseen sentences. All test cases are available for further experimentation in the GitHub repository.

The model performed well on straightforward examples:

Sentence	Model Guess
I did amazing in the exam today. Finally, I'll get good grades.	Positive
Today I got so tired, whole day I'm working like a dog. Sometimes I just want to sleep and do nothing.	Negative
The parliament of Georgia elected new Prime minister today.	Neutral
today was my first day at the university. It wasn't really good!	Negative
I can't say this wasn't one of the best experiences I've had.	Positive

This shows that:

- clearly emotional statements are correctly classified
- neutral factual statements are correctly identified

However, if we give the model slightly complex sentence, it shows its limitations:

```
text = ("I'm not unhappy with the results. quite the opposite.")  
print(classify_review(  
    text, model, tokenizer, device, max_length=train_dataset.max_length))
```

The model incorrectly classified this sentence as **negative**, even though the intended meaning is clearly **positive** (a double negation expressing satisfaction).

This example also highlights a common issue that is frequently faced in sentiment analysis, where the interpretation of linguistic aspects, such as negation, sarcasm, and even underlying sentiment, is a challenge for the model. These mistakes highlight that, even though the fine-tuned GPT-2 model has achieved strong overall performance, there is room for improvement.

6.6 Discussion and Conclusion

This work successfully demonstrates how a pretrained GPT-2 Medium model, that already embeds semantic information of a language, is a great tool and can be fine-tuned for a classification task. We built an architecture that supports an arbitrary number of classes and extended the methodology to a more realistic multi-class emotion detection problem.

One of the key observations from this study is the trade-off between efficiency and performance. We have seen that full fine-tuning yields slightly better performance at the expense of increasing the computational cost. Therefore, the partial fine-tuning proved to be the more efficient choice.

In conclusion, the implemented pipeline provides a solid foundation for similar projects and clearly illustrates the power of fine-tuning in natural language processing. All code and experiments are openly available in the accompanying GitHub repository for reproducibility and further extension.

7. Fine-tune to follow instructions

This project extends the instruction fine-tuning workflow from [1] into an academic writing assistant. The goal was not to build a general chatbot, but to adapt gpt2-medium into a focused prototype for three closely related writing tasks:

- grammar correction
- naturalness-oriented rewriting
- academic and professional rewriting

The final prototype was trained and evaluated through a sequential supervised fine-tuning pipeline using the checkpoints produced by the custom training path.

Importantly, these capabilities were not learned in one jointly mixed training run. They were learned through sequential checkpoint continuation: each later stage started from a previously fine-tuned checkpoint and adapted it to the next rewrite objective.

The primary objective was to adapt a pretrained GPT-2 model into a task-focused academic writing enhancement prototype, while preserving useful grammar-correction and naturalness-rewrite behavior.

This leads to 3 sub-goals:

1. A grammar-only stage should strongly outperform base GPT-2 on grammar correction.
2. A naturalness-rewrite stage, initialized from the Stage 1, using a combined dataset of naturalness pairs (AI-generated input with human-rewritten output) and grammar correction data should improve naturalness while retaining useful grammar behavior.
3. A final academic continuation stage, initialized from the Stage 2 checkpoint, should become the strongest checkpoint for academic rewriting.

7.1 Model and Training Setup

The base model for all experiments was gpt2-medium, therefore all parameters of configuration were in compliance to this model.

All stages used the same style instruction format:

Below is an instruction that describes a task. Write a response that appropriately completes the request.

Instruction:

...

Input:

...

Response:

...

Given an input sentence and a task instruction, the model is trained to generate the corresponding output sentence.

The project loads pretrained GPT-2 weights, applies LoRA adapters with rank 16 and alpha 32 (further explained in Appendix I), and trains using cross-entropy loss. Then it merges the LoRA weights into standard model.pth checkpoints for evaluation and downstream use. The smaller 768 block size is a training-memory choice, not a change to the model architecture.

The cross-entropy loss is computed only on the response tokens. Prompt tokens (instruction and input sections) are masked with the special ignore index -100 so that they do not contribute to the gradient. This helps the model learn the correct rewrite and not the prompt structure.

7.2 Dataset Gathering

The project used four distinct data sources:

- grammar dataset from the agentlans/grammar-correction [7]

instruction	input	output
Correct the grammar of the text.	Thanks you for your lovely comment Sue.	Thank you for your lovely comment Sue.
Correct the grammar of the text.	What gets for Karnal pincode?	How to get Karnal pincode?

- filtered naturalness rewrite pairs from dmitva/human_ai_generated_text [8]

instruction	input	output
Rewrite the text to sound more natural and fluent while preserving meaning.	When discussing the pros and cons of changing the school menu ...	The best way to provides healthy food for student is change ...

- GYAFC [9]

instruction	input	output
Rewrite the text in polished academic English.	Wow, I am very dumb in my observation skills	I do not have good observation skills
Rewrite the text in polished academic English.	usually i see him at my brothers ...	I usually see him with my brothers ...

- ParaSCI [10]

instruction	input	output
Rewrite the text in polished academic English.	twitter is a very popular micro blogging site.	twitter is a well-known social network service that allows users to post short 140-character status update which is called “tweet”.

The final plan is shown below.

Run	Dataset mix	Train	Validation	Test
Stage 1 Grammar	grammar only	10,200	600	1,200
Stage 2 Naturalness	naturalness pairs (5,200) + grammar (5,000) initialized from Stage 1	10,200	600	1,200
Stage 3 Academic	GYAFC + ParaSCI initialized from Stage 2	20,400	1,200	2,400

7.3 Sequential Training Design

We used **sequential checkpoint continuation** methodology, where a model is fine-tuned in a series of different stages. Rather than learning all capabilities in a single, jointly mixed training session, the model is adapted to new objectives one after another. Each stage begins with the weights from the previously fine-tuned checkpoint and further tunes them to a new task.

The Three-Stage Training Pipeline

1. **Stage 1: Grammar Specialization** – The base gpt2-medium model is first fine-tuned exclusively on error correction using a grammar dataset.
2. **Stage 2: Naturalness Adaptation** – Initialized from the Stage 1 checkpoint, this stage adds further training using the mix of naturalness and grammar datasets. The mixing is necessary due to poor grammar in only the naturalness dataset.
3. **Stage 3: Academic Writing Enhancement** – The final stage begins from the Stage 2 checkpoint and specializes the model in professional and academic rewriting using GYAFC and ParaSCI datasets.

Why Could This Design be Beneficial

Using a sequential design instead of a single multitask training offers several differences, which can be very interesting for the research:

- **Explicit Specialization:** we can observe each task separately and differentiate their requirements and results.
- **Transfer of Knowledge:** how the training from previous runs benefits the next phases.
- **Objective Analysis:** The sequential method can reveal the problem of “catastrophic forgetting”, meaning new tasks can overwrite parameters set by previous training. We can observe how significant this problem is and how can it be addressed.

7.4 Evaluation Protocol

The evaluation compared the three checkpoints (grammar, naturalness and academic) and the raw gpt2-medium outputs. Each model was evaluated on three respective dataset. \

The main metrics were:

- Cross-Entropy Loss and perplexity
- average token F1 between model output and reference
- BLEU and GLEU (read Appendix K for details)

While Loss measures the mathematical distance between the model's predictions and the reference, Perplexity evaluates the model's uncertainty level, where the lower the value, the higher the degree of confidence of the model in its predictions.

Token-Level F1 Score metric calculates the harmonic mean of precision and recall based on the overlap between generated and reference tokens. Lower scores indicate greater divergence from the reference text.

Additionally, to test how well the model manages to correct specifically the grammar, we used BLEU and GLEU scores. While BLEU shows overall quality, GLEU explicitly rewards successful edits and penalizes copying of errors.

7.5 Quantitative Analysis

Runtime and Loss

Total runtime across the training was approximately 60.7 minutes on Modal L40S. Both the train and validation losses were continuously decreasing, finally converged to the values given in the table below.

Run	Initialized from	Runtime (min)	Train loss	Validation loss
Stage 1 Grammar	base gpt2-medium	17.20	1.5376	1.5019
Stage 2 Naturalness	Stage 1 final checkpoint	18.57	2.2689	2.1900
Stage 3 Academic	Stage 2 final checkpoint	24.88	1.2897	1.2408

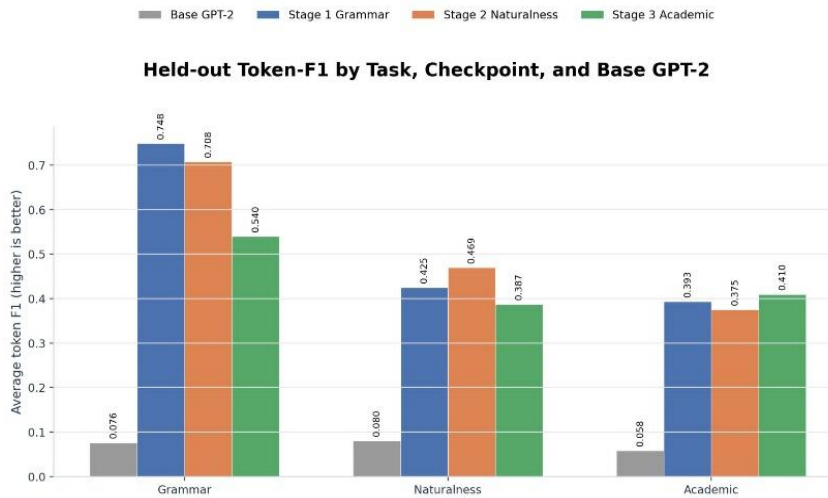
From the results of the training, we can notice that the model gradually adapted to new tasks. The increased value of validation loss in stage 2 (2.1900) indicates that the task of the model's adaptation to naturalness was much more complicated and subjective than any other rule-based task. Validation loss in stage 3 is 1.2408, which means that the model succeeded in learning to paraphrase academic and scientific texts based on its earlier experience. Besides, in all stages of training, the value of validation loss remains lower than the training one, which indicates a good generalization and shows that the model learned useful patterns without overfitting to the training data.

Grammatical Evaluation Analysis (GLEU: 72.56 | BLEU: 60.88)

The accuracy of the model in grammatical editing tasks demonstrates a strong ability of correcting targeted mistakes while maintaining the structure of the sentence.

- The GLEU score of 72.56 shows that although the outputs are not perfectly accurate, in most cases the target errors have been corrected successfully.
- The BLEU score of 60.88 implies significant overlap between the generated and the reference texts, meaning the model corrects the grammar without diverging too much from the input.

Overall token-F1 comparison (detailed explanation of the measurement in Appendix J)



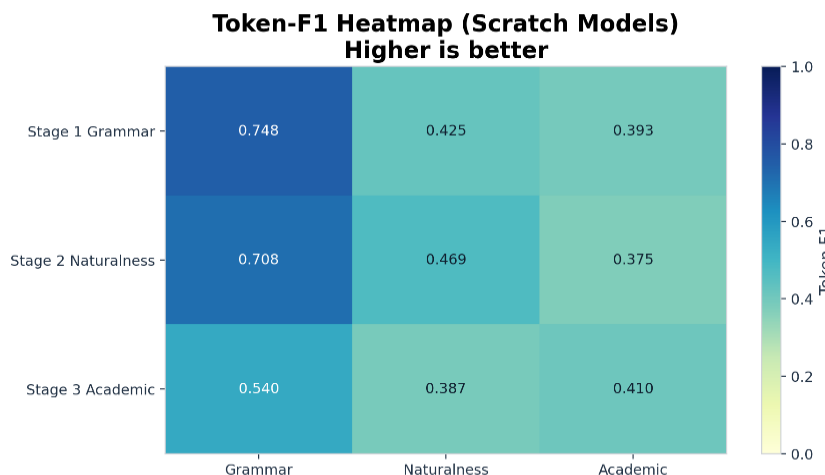
Token-F1 chart compares accuracy of the Base GPT-2 model and our fine-tuned checkpoints on the Grammar, Naturalness and Academic rewriting tasks.

Bar height represents the Token-F1 score, measuring the overlap between generated and reference text. taller bars indicate higher accuracy.

The Base GPT-2 model scores almost zero in all the tasks (0.058 to 0.080). Turns out, while it technically can generate English sentences, it completely fails to follow the specific task requirements.

The F1 chart gave us important insights into training progresses. When the model moves from Stage 1 to Stage 2, its Grammar F1 drops slightly from 0.748 to 0.708 and moving to Stage 3 it falls to 0.540. So does Naturalness accuracy from 0.469 to 0.387. This is already a sign that new tasks adapt previously set parameters for themselves.

Besides, Academic rewriting turned out to be the hardest to master, it remained low until Stage 3 directly addressed it and slightly increased to 0.410.



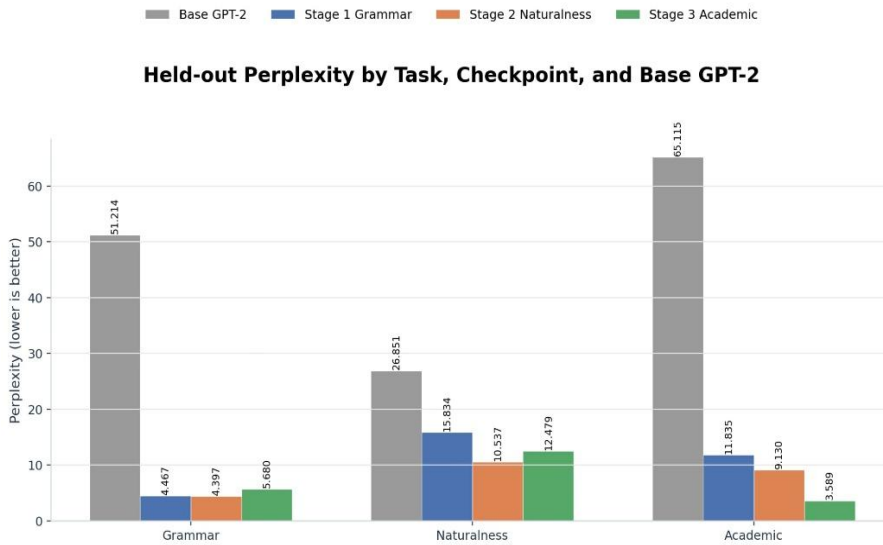
The Heatmap clearly shows that each checkpoint peaks specifically on the task it was actively trained for.

Grammar is strongest in **Stage 1** (0.748).

Naturalness is strongest in **Stage 2** (0.469).

Academic is strongest in **Stage 3** (0.410).

Perplexity comparison (detailed explanation of the measurement in Appendix J)



The four models were evaluated on Grammar, Naturalness, and Academic tasks using the provided instructions and inputs.

The Perplexity chart shows how training on one task affects performance on others, revealing that during specialization knowledge transfer comes with minor regressions.

Bar height reflects model Perplexity. In simple terms, it reflects how “surprised” the model is by the correct text. The shorter the bar - the less surprising and more predictable the output.

Stage 1 (Grammar): Grammar training leads to a significant performance rise, the grammar perplexity becomes very low, only 4.467. In addition, it also makes our model much better in naturalness and academic writing already, without direct training.

Stage 2 (Naturalness): The model obtains the best results in naturalness perplexity - 10.537. With grammar replay, naturalness and academic perplexity also improve (4.397 for grammar and 9.130 for academic), making this the most balanced checkpoint overall.

Stage 3 (Academic): In academic specialization, we obtain the lowest perplexity score ever, 3.589. At the same time, some regressions appear due to the deep specialization, rising naturalness perplexity to 12.479 and grammar to 5.680.

Understanding the limitation

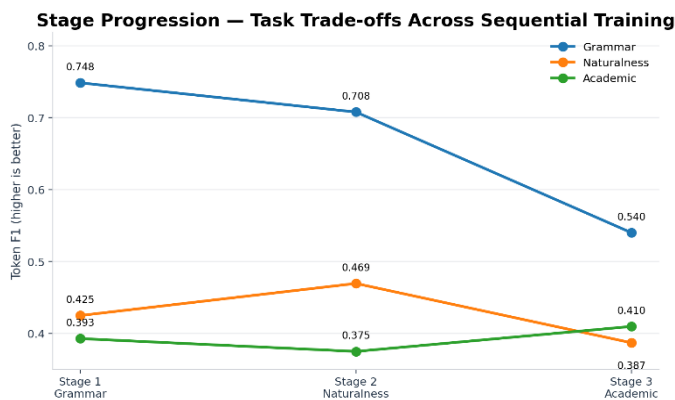
The naturalness and academic token-F1 scores are moderate rather than high for several reasons.

- Every task except grammar correction is nondeterministic and very hard to address with small dataset.
- As expected, the naturalness dataset was noisier than the grammar and academic and some references were weak semantic targets.
- Clearly, it's not only the problem of the dataset but GPT-2-medium also has limited instruction-following capacity compared to modern models.
- Finally, there is problem in our decision too: due to sequential design, later stages adapt the model toward the newest objective and partially overwrite earlier task behavior.

For these reasons, we can even say that achieving the mid-range scores was a success for the research.

Task trade-offs

Useful finding in this section is that with training not all the tasks improve together.



Stage 1 improves grammar and partially helps academic rewriting.

Stage 2 becomes the strongest naturalness specialist while retaining much of Stage 1's grammar behavior.

The final Stage 3 improves academic rewriting, while discarding some grammar and naturalness specialization.

This pattern is important because it demonstrates how sequential instruction fine-tuning works: later stages create useful specialization but also reduce performance on earlier task objectives.

According to [11], deterioration in the performance on the previous tasks after tuning at the next stage is considered an expected result of the sequential design and is known as **catastrophic forgetting** or **objective interference**.

7.6 Manual Testing and Qualitative Results

To assess our model in real life situations and how useful it really is, we tested stage 3 checkpoint manually.

Grammar Correction Task

Grammar could be tested with the instruction: *“Correct the grammar of the text”*.

As already mentioned, error correction is quite deterministic and needs accuracy more than creativity, so temperature of 0.0 (greedy decoding) was chosen for this task.

Original Input	Model Correction
He don't know nothing about math.	He doesn't know anything about math.
I am used to wake up early.	I am used to waking up early.
If I would have more time I will probably learn another language.	If I had more time I would probably learn another language.
The more you practice, the better you will become it.	The more you practice, the better you will become.
Me and my friend we went to cafe and we was talking about different things for hours	Me and my friend we went to cafe and we were talking about different things for hours.

This model proves its proficiency in applying basic grammatical rules, such as:

- correcting double negative: “don't know nothing” → “doesn't know anything”,
- correcting gerundive form: “wake” → “waking”
- correct conditional verbs: “would have... will” → “had... would”
- recognized and removed the extra pronoun in the end of the fourth sentence.

However, the last example proves the limitations of the model. Although it managed to correct the mistake of agreement of subject with the verb (“we was” → “we were”), it did not manage to correct the informal phrases “Me and my friend” and omission of the article before “to a cafe”.

Naturalness Rewriting Task

Naturalness could be tested with the instruction: *“Rewrite the text to sound more natural and fluent while preserving meaning.”*

In order to avoid repetition of the original text and let the model use its power of creativity, we considered the temperature of 0.4 the most suitable for this task.

Original Input	Model Correction
As I walked to the store, I noticed an unmistakable puff of chill frost crust covering the surface of the door.	I walked to the store and noticed an unmistakable puff of chill frost crust covering the surface of the door.
Finally, the children started singing. The children danced vividly too	The children started singing and dancing.

These examples demonstrate successful performance of the Naturalness task. The model simplifies the first sentence by breaking the dependent clause, while in the second sentence it combines two disjointed sentences, giving more natural flow.

Academic Rewriting Task

Academic rewriting could be tested with the instruction: *“Rewrite the text in polished academic English.”*

For this kind of task, we tried different temperatures ranging from 0.0 to 0.7.

Original Input	Model Correction	Temp
The experiment result show significant improvements.	The results show significant improvements in the accuracy of the subjects' responses.	0.4
This paper try to analyze how social media affect students performance.	This paper analyze how social media affect students performance.	0.0
We did a bunch of tests and the numbers show that our approach is the best so far.	We performed a bunch of tests and the results showed that the approach we used is the best so far.	0.7

Both, the model's strengths and weaknesses can be seen from the academic rewriting results.

- **Lexical improvement:** The model does an excellent job of using more academic vocabulary, such as "performed" instead of "did," and "results" instead of "numbers."
- **Hallucination:** In the first case, the model fixes the grammar but provides additional details not mentioned in the original text, focusing on an academic style rather than accuracy. It is one of the typical side effects of using small models such as GPT-2 Medium.
- **Catastrophic forgetting:** In the second case, the model makes the phrasing better but makes grammatical mistakes.

7.7 Specialization VS Generalization

To better evaluate the sequential design approach, we separately trained the same base gpt2-medium model using joint multi-task approach. We mixed grammar, naturalness, and academic examples in one training dataset, but changed nothing else.



The chart compares average F1 scores across three tasks (Grammar, Naturalness, and Academic) for three approaches:

Blue (Best specialist): The average of the highest-scoring models for each task (Stages 1, 2, and 3, respectively).

Orange (Stage 3): The average across all tasks using only the Stage 3 model.

Green (Joint multi-task): The average across all tasks using the Joint model.

As expected, the joint result does not surpass the best specialist for each task. Moreover, it gives only slightly better average results, which shows the limitation factor was more of a dataset quality and weak hardware, rather than the design itself.

8. Conclusion

The main goal of this thesis was to first of all explore and then provide a comprehensive, practical as well as theoretical insights about building a Large Language Model from scratch.

This project covered several architectural milestones:

- **Data Preprocessing:** We integrated Byte Pair Encoding (BPE) tokenizer and constructed a custom sliding window data loader.
- **Causal Attention:** We coded a multi-head causal self-attention mechanism to calculate multiple semantic and syntactic relationships in parallel and also added causal masking to make sure model only looks at past words when predicting the next one.
- **Pre-training:** We implemented a training loop using optimization technique of AdamW with learning rate warmup and cosine decay.
- **Supervised Sentiment Classification:** We added our own custom classification head to frozen pre-trained layers and measured efficiency vs effectiveness trade-offs.
- **Sequential Instruction Fine-Tuning:** We fine-tuned the model and created an academic writing assistant using Low-Rank Adaptation (LoRA) in three different stages (Grammar, Naturalness, and Academic Style).

We started with raw text and math and finished with a working neural network capable of predicting sentiment and rewriting texts in academic style. We showed that while the massive data centers may be better suited for general-purpose AI, building useful, specialized LLMs with limited resources is also absolutely possible.

8.1 Observations: Efficiency VS Effectiveness

Our work has provided valuable insights into the behavior of decoder-only architectures.

Adaptations for Classification

Our experiments on Twitter sentiment datasets proved that using pre-trained language models with partial parameter freezing is highly efficient. With updating only the final layer normalization, the final transformer block and the custom classification head, the model successfully achieved 74.54% accuracy on Twitter sentiment data in just under 23 minutes. With this we concluded that a highly efficient architecture can still build a highly effective model.

Sequential LoRA Training

During instruction fine-tuning we used LoRA adapters, which let us save computational resources by training only part of the parameters at each step, instead of retraining the entire huge model. Despite the fact that sequential training led to severe catastrophic forgetting, the LoRA happened to be not only algorithm for efficiency, but also an approach to avoid total knowledge loss as untrained part allowed the model to maintain past skills while learning new ones.

8.2 Limitations of the Current Prototype

Our results demonstrate the architectural power of generative transformers. However, some technical and physical constraints have to be considered:

- **Dataset Quality:** As the primary data collection was out of the scope of this thesis, the study relied on the existing public datasets. While highly accessible, this introduced certain limitations regarding quality.
- **Physical Hardware:** Due to physical hardware limitations, maximizing computational efficiency was critical. This required aggressive optimization techniques and reducing size of the datasets.
- **Model Capacity and Hallucinations:** Standard GPT-2 Medium (355M parameters) has limited capacity compared to modern massive architectures, resulting in minor semantic hallucinations.

8.3 Final Summary

In conclusion, this project made a connection between theory and practice, proved that the Large Language Models can be constructed without supercomputers. The main achievement of this thesis is that it exactly describes the process of efficiently developing an LLM step by step, which results in a model that can be eventually used in real-world applications.

Bibliography

- [1] S. Raschka, *Build a Large Language Model (From Scratch)*. Shelter Island, NY, USA: Manning Publications, 2024.
- [2] E. Wharton, “The Verdict,” *Wikisource, the free online library*. [Online]. Available: https://en.wikisource.org/wiki/The_Verdict
- [3] PyTorch, “torch.nn.Embedding — PyTorch documentation,” 2025. [Online]. Available: <https://pytorch.org/docs/stable/generated/torch.nn.Embedding.html>
- [4] A. Vaswani et al., “Attention Is All You Need,” in *Proc. 31st Conf. Neural Inf. Process. Syst. (NIPS)*, Long Beach, CA, USA, 2017.
- [5] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” *arXiv preprint arXiv:1711.05101*, 2017.
- [6] A. Shrivastava, “Sentiment Analysis Dataset,” Kaggle, 2020. [Online]. Available: <https://www.kaggle.com/datasets/abhi8923shriv/sentiment-analysis-dataset>
- [7] Agentlans, “Grammar Correction Dataset,” Hugging Face, 2023. [Online]. Available: <https://huggingface.co/datasets/agentlans/grammar-correction>
- [8] Dmitva, “Human AI Generated Text Dataset,” Hugging Face, 2023. [Online]. Available: https://huggingface.co/datasets/dmitva/human_ai_generated_text
- [9] S. Rao and J. Tetreault, “Dear Sir or Madam, May I Introduce the GYAFC Dataset: Corpus, Benchmarks and Metrics for Formality Style Transfer,” in *Proc. NAACL-HLT*, 2018, pp. 129–140. [Online]. Available: <https://aclanthology.org/N18-1012.pdf>
- [10] Q. Dong, X. Wan, and Y. Cao, “ParaSCI: A Large Scientific Paraphrase Dataset for Longer Paraphrase Generation,” in *Proc. EACL*, 2021, pp. 424–434. [Online]. Available: <https://aclanthology.org/2021.eacl-main.33.pdf>
- [11] R. Kemker, M. McClure, A. Abitino, T. Hayes, and C. Kanan, “Measuring Catastrophic Forgetting in Neural Networks,” *AAAI*, vol. 32, no. 1, Apr. 2018.
- [12] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, “BLEU: a method for automatic evaluation of machine translation,” in *Proc. 40th Annu. Meeting Assoc. Comput. Linguistics*, 2002, pp. 311–318.

Appendices

Appendix A: Technical Specifications.

All code implementations developed for this project are available in the GitHub repository <https://github.com/GiorgiGogsadze/LLM-From-Scratch-Thesis>

This thesis builds upon the approach described in [1], extending it by applying the methodologies to a different problem.

Most of the Model Architectural code presented in this work is based on [1], with modifications and improvements introduced for the purposes of this study.

The dataset used in classification fine-tuning was obtained from Kaggle [6] and is distributed under a CC0 (public domain) license, allowing unrestricted use for research and educational purposes.

The datasets used in Instruction fine-tuning were obtained from Hugging Face [7], [8] and official research repositories [9], [10]. These corpora are derived from publicly available sources and are released by their respective authors for non-commercial research and educational purposes.

Appendix B: Transformer Architecture

The original **Transformer** model's basic structure consists of two parts: an **encoder** and a **decoder**. As can be seen in figure B.1, the encoder reads the input text and converts it into a vector representation. It encodes its contextual information. The decoder then decodes this numerical representation to generate the output text. One of the major uses of this model is a translation application, whereby the encoder would process the source language text, and the decoder would generate the translated text in the target language.

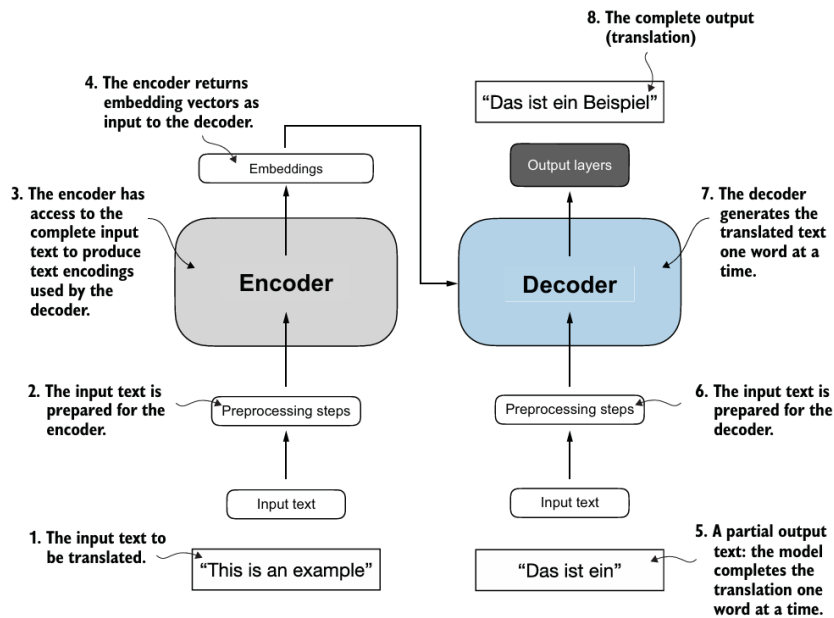


Fig. B.1: Transformer Architecture (Source: [1], p. 8)

Moreover, the self-attention mechanism is one of the key features of the transformer architecture. The mechanism is employed to provide weights for different words so that the model can acquire information regarding dependencies and word relations.

The **GPT (Generative Pre-trained Transformer)** architecture simplifies this design. It utilizes **only the decoder** portion of the original Transformer. This focus on the decoder allows them to predict next word in a sequence, which is the core of text generation.

Appendix C: Byte Pair Encoding (BPE)

BPE is a data compression algorithm adapted for text tokenization. Instead of storing entire words, it builds a vocabulary of characters and frequently occurring subwords.

The main idea:

- Start with individual characters.
- Repeatedly merge the most frequent pair of tokens (characters or subwords) into a new token.
- Continue until you reach a target vocabulary size.

This produces a compact vocabulary where common words become single tokens, and rare/unseen words are represented as a sequence of subwords or characters.

BPE is particularly useful because:

1. **Common words become single tokens.** Example: “the” or “because”.
2. **Rare or unseen words are split,** no need for <|unk|>

Example:

- “unhappiness” → [un, happiness]
- “Akwirw” → [Ak, w, ir, w]. Shown in figure C.1.

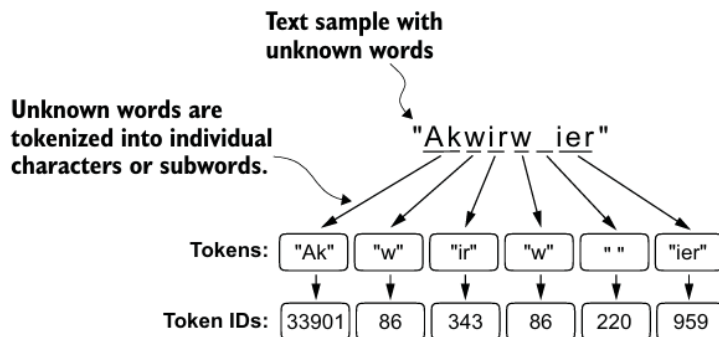


Fig. C.1: BPE tokenizer visualization
(Source: [1], p. 34)

Appendix D: torch.nn.Embedding

*“The **torch.nn.Embedding** class in PyTorch provides a simple lookup table that stores embeddings of a fixed dictionary and size. This module is often used to store word embeddings and retrieve them using indices. The input to the module is a list of indices, and the output is the corresponding word embeddings.”*

explanation from [3]

In this project, both token embeddings and positional embeddings are implemented using PyTorch’s torch.nn.Embedding. This class provides a lookup table that maps indices to trainable vectors.

For **token embeddings**, each token ID is mapped to a vector of a specified size, forming a weight matrix that is updated during training.

For **positional embeddings**, a similar lookup table is created, where each row corresponds to a position in the input sequence. The input positions are represented as a sequence of indices, and nn.Embedding retrieves the vector at each position. these vectors are also learned during training.

By adding token embeddings and positional embeddings, the model receives information about both the identity and the order of tokens.

Appendix E: The Architecture of Query, Key, and Value Representations

To ensure the model learns how to interpret embeddings rather than comparing them directly, we employ a strategy of projecting input embeddings into three distinct learned representations:

- **Query (q_i)** — represents what the current token is *looking for*
- **Key (k_i)** — represents how each token can be *matched*
- **Value (v_i)** — represents the *information content* of each token

To get these representations, three different weight matrices (W_Q , W_K and W_V) are initialized with **random values** and optimized during training using gradient descent. Each token embedding x_i is then projected as:

$$q_i = x_i W_Q, \quad k_i = x_i W_K, \quad v_i = x_i W_V$$

As explained in [4], query, key, and value projections are often performed in lower-dimensional spaces, particularly in multi-head attention, where each head operates on a reduced representation. Therefore, even here we reduce output dimension d_{out} as:

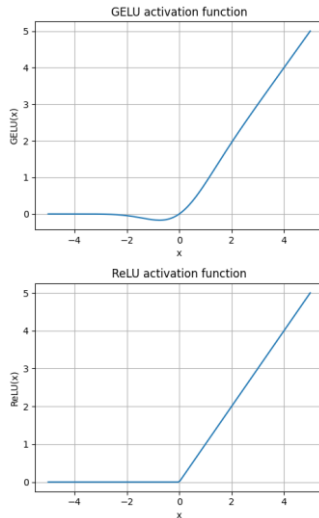
- It reduces computational cost,
- It enables multiple attention heads to operate in parallel,

It allows the model to learn compact, task-relevant representations.

Appendix F: GELU VS RELU

The **ReLU (Rectified Linear Unit)** activation used to be adopted widely, because of its simplicity and computational efficiency. However, GELU is a smoother activation function with superior optimization behavior during training and has been shown to enhance the performance of deep transformer architectures.

The comparison of both of these functions is shown below:



In the plot we can see that ReLU outputs a zero for negative inputs and is linear for positive values, while GELU is smooth nonlinear function which gives small outputs even for negative inputs. This smooth behavior has several advantages.

- Offers smooth transitions, which is good for optimization.
- Keeps neurons active on negative inputs, allowing more stable and richer gradients.

For these reasons, GELU has become a de-facto standard activation function in GPT and other large language models.

Appendix G: Calculating the Loss

Loss calculation follows the steps:

1. When GPT model processed a sequence, the network outputs **logits**, which are numerical scores representing the model's confidence for each token in the vocabulary.
2. After softmax, we can look at the **probability** of all 50,257 tokens.
3. A good prediction is one that has a high probability for the right tokens. Ideally, the model should give a probability close to **1** for the correct token and close to **0** for others. But before training the model usually gives almost the same probability to every token in the vocabulary, so it acts like random guessing.
4. To evaluate the quality of the prediction, we compute the **logarithm of the probability** assigned to the correct token. Logarithm makes it easier to see how good the prediction is. When the correct token has high probability, the logarithm is close to 0. If the probability is very small, the logarithm is a large negative number, meaning the prediction is bad.
5. When training a language model, we are not just predicting a **single token**. Each training example contains many tokens. Instead of storing all these values separately, we compute their average. Thus, we have one single number that indicates how well the model predicted the whole sequence.
6. Since probabilities are between **0 and 1**, their logarithms are always **negative or zero**. Multiplying the result by **-1** provides a positive value that can be minimized during training.

The resulting value indicates how far the predictions of the model are from the correct tokens. In practice, these steps are combined into a single metric called **cross-entropy loss**.

Appendix H: AdamW Optimizer

AdamW optimizer is a common standard for training transformer models.

A parameter w is updated as

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \text{gradient}$$

- w_{old} is the current value of a model parameter (weight).
- w_{new} denotes the updated parameter value.
- η is the learning rate, which controls the step size of the update.
- **gradient** represents the derivative of the loss function with respect to w_{old} , i.e.,

$$\text{gradient} = \nabla_w \mathcal{L}(w_{\text{old}})$$

But applying the same learning rate to all parameters can lead to slow training or instability. Adam improves on this by automatically adapting the learning rate for each parameter, depending on how the gradients performed in prior training steps. This enables the model to learn faster and more reliably.

AdamW is a variant of Adam with **decoupled weight decay**. As explained in [5], weight decay slightly reduces the size of the model's weights during training:

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \text{gradient} - \eta \lambda w_{\text{old}}$$

- λ is the weight decay coefficient (set to 0.1 in this implementation), controlling how aggressively weights are penalized for growing large.

This prevents the weights from becoming too large, which helps to avoid **overfitting** and helps the model to generalize better to new data.

Because of its stability, adaptive learning rates, and built-in regularization, AdamW has become a standard optimizer for training transformer-based models.

Appendix I: Parameter-Efficient Fine-Tuning with Low-Rank Adaptation

Low-Rank Adaptation (LoRA) is an advanced, highly popular technique for parameter-efficient fine-tuning. While the baseline approach to fine-tuning involves updating all of the parameters within the network, LoRA adapts a pretrained model to a specific task by adjusting only a small, focused subset of the model's overall weight parameters.

In traditional full fine-tuning, during the backpropagation step, the network learns a massive weight update matrix (ΔW) that matches the exact dimensions of the original pretrained weight matrix (W), such that the new weights are calculated as

$$W_{\text{updated}} = W + \Delta W.$$

LoRA bypasses this computational bottleneck by keeping the original pretrained weights frozen and approximating the update matrix using two significantly smaller matrices, A and B , such that $\Delta W \approx AB$. The dimensions of these matrices are less than a hyperparameter called the inner dimension or rank (r). With this “low-rank” approach, model’s computations stay within smaller dimension and successfully captures only the most influential changes in weight parameters.

The rank determines the shape and size of the matrices, but there is another hyperparameter alpha (α), which controls how strong their impact should be on the original model - how much the newly learned LoRA parameters influence the original parameters.

When the LoRA matrices are merged with the base model, the calculation is not just $W + BA$. The updates are scaled by a factor of $\frac{\alpha}{r}$. So, the true formula is:

$$W_{\text{new}} = W + \frac{\alpha}{r} \Delta W$$

LoRA optimizes the fine-tuning of large language models by drastically reducing computational and memory demands, enabling scalable storage with lightweight, swappable weights, and preventing catastrophic forgetting by preserving foundational knowledge.

Appendix J: Token-F1 & Perplexity

Token-F1 measures the exact overlap between the model's generated text and the reference text. To calculate this measure, it is necessary to determine three values by comparing the model's generated tokens against the target tokens:

- **True Positives (TP):** Number of tokens generated by the model that match the exact tokens in the target text.
- **False Positives (FP):** Number of extra or incorrect tokens generated by the model that are not present in the target text.
- **False Negatives (FN):** Number of target tokens that the model failed to generate.

Based on these values we can calculate **Precision (P)**, which measures the quality of the model's output (how many of the generated tokens were correct):

$$Precision(P) = \frac{TP}{TP + FP}$$

And **Recall (R)**, which measures the coverage of the model's output:

$$Recall(R) = \frac{TP}{TP + FN}$$

the Final formula for F1 score is the **harmonic mean** of Precision and Recall (how many of the target tokens the model managed to capture):

$$F_1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

The harmonic mean is used here, instead of simple average. This approach strictly penalizes extreme imbalances, ensuring that the final Token F1 score is only high if the model achieves both high precision (the quality of the model's output) and high recall (the coverage of the model's output) simultaneously.

Perplexity measures how well a probability model predicts a sample a lower perplexity means the model is less surprised by the text and predicts the next tokens with higher confidence.

perplexity is calculated directly from the **Cross-Entropy Loss**:

$$PPL = e^{\text{Loss}}$$

Based on this formula, Perplexity of 1 would mean the model predicts the exact correct next token with 100% certainty every single time, while $n > 1$ means it had to choose uniformly at random between n valid words.

Appendix K: BLEU & GLEU

BLEU (Bilingual Evaluation Understudy) measures the exact overlap between a generated sequence and a reference. It breaks sentence into tiny chunks of 1 to 4 words, counts how many chunks does exactly match the reference, but counts one correct word only once. It also checks the length and reduces score if candidate output is way shorter than the reference. Exact mathematical definition is out of scope for this thesis, please refer to [12] for more information.

However, BLEU heavily rewards models that safely copy the source text word for word. This results in artificially high scores for models that fail to actually correct errors.

GLEU (Generalized Language Evaluation Understanding)

To address BLEU's mathematical flaws, **GLEU** incorporates three data points: the candidate output (C), the reference target (R), and the erroneous source text (S).

Instead of reading the whole sentence at once, GLEU looks at small 1 to 4 tokens at a time. If the original sentence had a mistake, and the model correctly fixed it, points are increased. If the model copied a mistake from the original sentence without fixing it, points are decreased. The final score bounds the output strictly between 0 and 1 by taking the minimum of the adjusted precision and recall:

$$\begin{aligned} \text{Pen} &= |C \cap S \setminus R| \\ \text{Precision} &= \frac{|C \cap R| - \text{Pen}}{|C|} \\ \text{Recall} &= \frac{|C \cap R| - \text{Pen}}{|R|} \\ \text{GLEU} &= \max(0, \min(\text{Precision}, \text{Recall})) \end{aligned}$$

A model cannot achieve a high GLEU score through passive copying. It must actively and accurately edit the text.